

LAB FILE

OBJECT ORIENTED PROGRAMMING PRACTICES

UNC302

NAME: MEDHACHHAVI DHAWAN

ROLL NUMBER:1024150216

SUBMITTED TO : Mr VARUN SODHI



THAPAR INSTITUTE
OF ENGINEERING & TECHNOLOGY
(Deemed to be University)

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING
THAPAR INSTITUTE OF ENGINEERING AND TECHNOLOGY, (A DEEMED TO BE
UNIVERSITY), PATIALA, PUNJAB

INDIA

Session-Year(Jan-May, 2026)

INDEX

<u>S no.</u>	<u>EXPERIMENT</u>
1	Lab Assignment:1
2	Lab Assignment:2
3	Lab Assignment:3
4	Lab Assignment:4
5	Lab Assignment:5
6	Lab Assignment:6
7	Lab Assignment:7
8	Lab Assignment:8
9	Lab Assignment:9

LAB EXPERIMENT 1

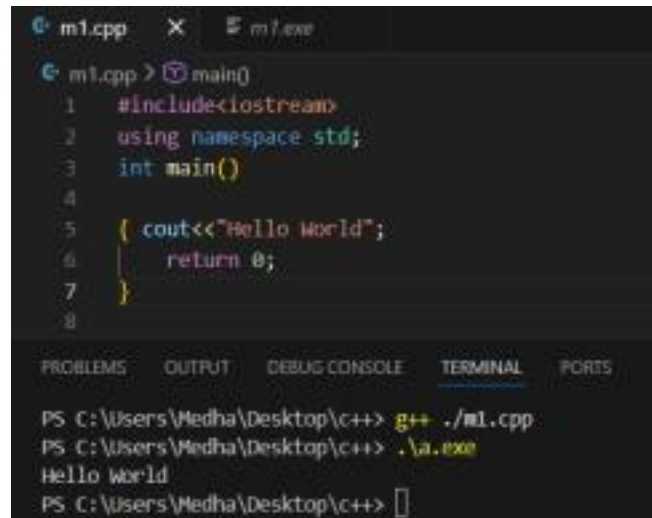
1. Write a program (WAP) to display "Hello World" on console display.

```
#include<iostream>

using namespace std;

int main()
{
    cout<<"Hello World";

    return 0;
}
```



The screenshot shows a C++ IDE with a file named m1.cpp. The code is as follows:

```
m1.cpp > main()
1  #include<iostream>
2  using namespace std;
3  int main()
4  {
5      cout<<"Hello World";
6      return 0;
7  }
```

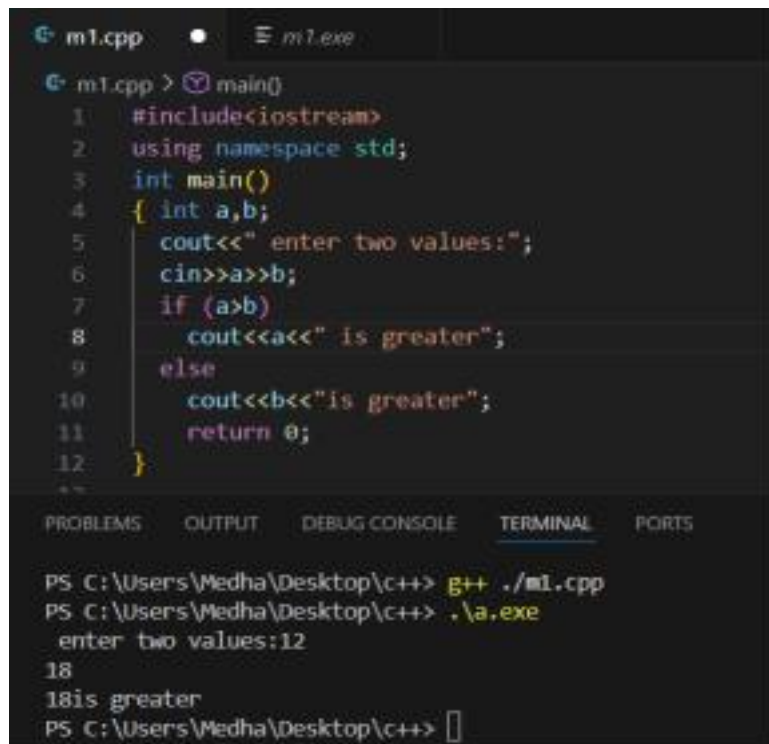
The terminal output shows the command `g++ ./m1.cpp` and `./a.exe` being executed, resulting in the output `Hello World`.

2. WAP to that reads two values from keyboard and displays the larger value on the screen.

```
#include<iostream>

using namespace std;

int main()
{
    int a,b;
    cout<<"enter two values:";
    cin>>a>>b;
    if(a>b)
        cout<<a<<" is greater";
    else
        cout<<b<<" is greater";
    return 0;
}
```



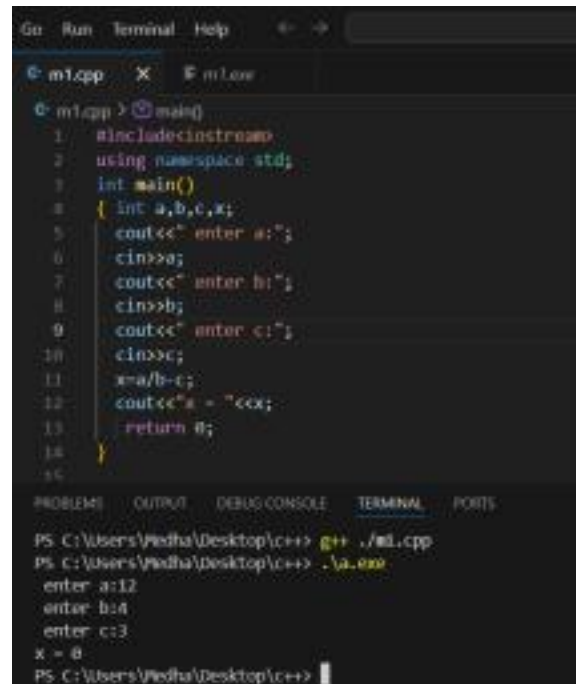
The screenshot shows a C++ IDE with a file named m1.cpp. The code is as follows:

```
m1.cpp > main()
1  #include<iostream>
2  using namespace std;
3  int main()
4  { int a,b;
5      cout<<" enter two values:";
6      cin>>a>>b;
7      if (a>b)
8          cout<<a<<" is greater";
9      else
10         cout<<b<<" is greater";
11         return 0;
12     }
```

The terminal output shows the command `g++ ./m1.cpp` and `./a.exe` being executed. The user enters `12` and `18` as input, and the output is `18 is greater`.

3. WAP to read three values a, b, c from keyboard and display the value of $x = a/b - c$.

```
#include<iostream>
Using namespace std;
Int main()
{ int a,b,c,x;
Cout<<"enter a:";
Cin>>a;
Cout<<"enter b:";
Cin>>b;
Cout<<"enter c:";
Cin>>c;
x=a/b-c;
cout<<"x = "<<x;
return 0;
}
```



The screenshot shows a C++ IDE with a file named m1.cpp. The code in the editor is as follows:

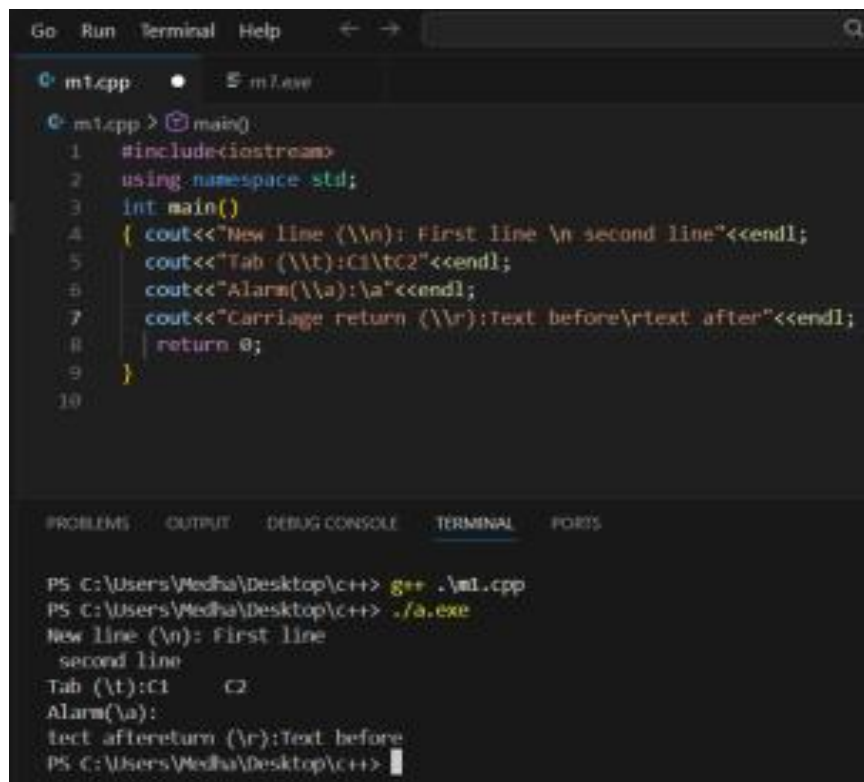
```
1 #include<iostream>
2 using namespace std;
3 int main()
4 { int a,b,c,x;
5   cout<<" enter a:";
6   cin>>a;
7   cout<<" enter b:";
8   cin>>b;
9   cout<<" enter c:";
10  cin>>c;
11  x=a/b-c;
12  cout<<"x = "<<x;
13  return 0;
14 }
```

The terminal output shows the execution of the program:

```
PS C:\Users\Medha\Desktop\c++> g++ ./m1.cpp
PS C:\Users\Medha\Desktop\c++> ./a.exe
enter a:12
enter b:4
enter c:3
x = 0
PS C:\Users\Medha\Desktop\c++>
```

4. WAP to implement the following control characters:

- '\n' is for new line, or you can use endl – cout<<endl<<"message";
- '\t' is for tab
- '\a' is an alarm sound
- '\r' is carriage return to go to the beginning of the current line



The screenshot shows a C++ IDE with a file named m1.cpp. The code in the editor is as follows:

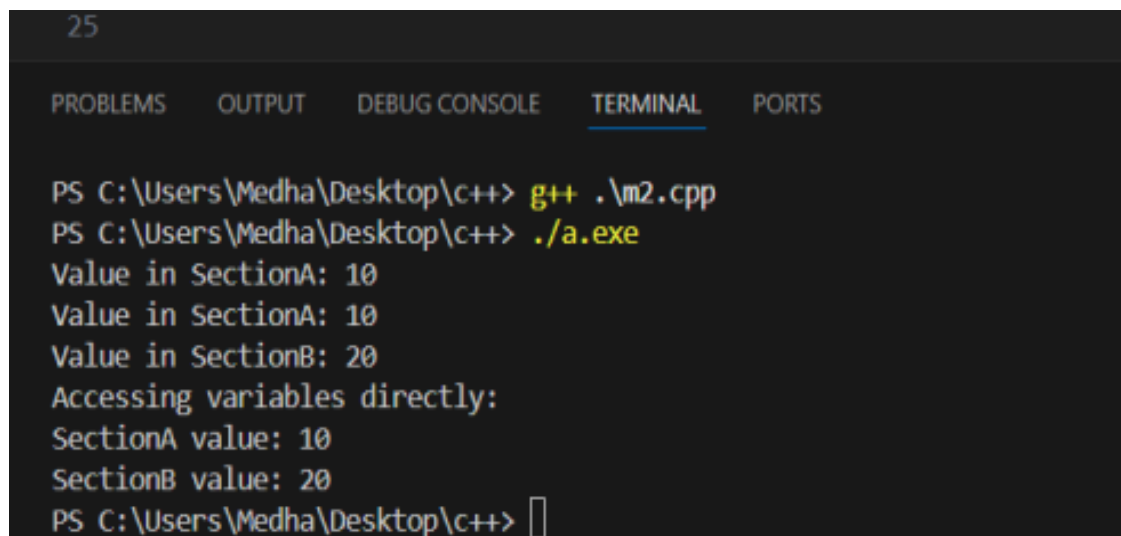
```
1 #include<iostream>
2 using namespace std;
3 int main()
4 { cout<<"New line (\n): First line \n second line"<<endl;
5   cout<<"Tab (\t): C1\tC2"<<endl;
6   cout<<"Alarm (\a): \a"<<endl;
7   cout<<"Carriage return (\r): Text before\rtext after"<<endl;
8   return 0;
9 }
10
```

The terminal output shows the execution of the program:

```
PS C:\Users\Medha\Desktop\c++> g++ ./m1.cpp
PS C:\Users\Medha\Desktop\c++> ./a.exe
New line (\n): First line
second line
Tab (\t): C1      C2
Alarm (\a):
text afterreturn (\r): Text before
PS C:\Users\Medha\Desktop\c++>
```

5. Implement *namespace* in a program to illustrate the use of same name variables and functions in different sections/libraries of the code.

```
#include <iostream>
using namespace std;
namespace SectionA {
    int value = 10;
    void display() {
        cout << "Value in SectionA: " << value << endl;
    }
}
namespace SectionB {
    int value = 20;
    void display() {
        cout << "Value in SectionB: " << value << endl;
    }
}
int main() {
    SectionA::display();
    SectionB::display();
    cout << "Accessing variables directly:" << endl;
    cout << "SectionA value: " << SectionA::value << endl;
    cout << "SectionB value: " << SectionB::value << endl;
    return 0;
}
```

A screenshot of a terminal window with a dark background. At the top left, the number '25' is displayed. The terminal has tabs for 'PROBLEMS', 'OUTPUT', 'DEBUG CONSOLE', 'TERMINAL' (which is selected and underlined), and 'PORTS'. The command prompt shows the user is in the directory 'C:\Users\Medha\Desktop\c++'. The commands entered are 'g++ .\m2.cpp' and './a.exe'. The output of the program is displayed line by line: 'Value in SectionA: 10', 'Value in SectionA: 10', 'Value in SectionB: 20', 'Accessing variables directly:', 'SectionA value: 10', 'SectionB value: 20'. The prompt returns to 'PS C:\Users\Medha\Desktop\c++>' with a cursor.

6. Write a program to define a structure *Student* that contains 3 variables and 2 functions:

- Variables:

(a) ***name***

(b) ***roll***

(c) ***marks***

- Functions:

(1) ***setStudentData()***

(2) ***getStudentData()***

Use loops to input and output data for 3 student object entries.

```

#include<iostream>
#include<string>
using namespace std;
struct student {
    string name;
    int roll;
    float marks;
    void setStudentData(string n, int r, float m)
    { name = n;
      roll = r;
      marks= m;}
    void getStudentData()
    { cout<<"Name : "<<name <<"roll no : "<< roll<<" marks : "<<marks;} };
int main()
{ student s3[3];
  string n;
  int r;
  float m;
  for(int i=0;i<3;i++)
  {cout<<"enter details of student "<<i+1;
   cout<<"Name: ";cin>>n;
   cout<<"Roll no.: ";cin>>r;
   cout<<"Marks: ";cin>>m;
   s3[i].setstudentData(n,r,m);}
  cout<<"\n Displaying student details:\n"; for(int
  i=0;i<3;i++)
  {
  s3[i].getStudentData();
  }
  return 0;
}

```

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\Medha\Desktop\c++> g++ .\m1.cpp
PS C:\Users\Medha\Desktop\c++> ./a.exe
enter details of student 1Name: Amit
Roll no.: 1025030123
Marks: 89
enter details of student 2Name: Rohit
Roll no.: 1025060145
Marks: 98
enter details of student 3Name: Ankit
Roll no.: 1025050234
Marks: 90

Displaying student details:
Name: Amit
Roll number: 1025030123
marks: 89
Name: Rohit
Roll number: 1025060145
marks: 98
Name: Ankit
Roll number: 1025050234
marks: 90
PS C:\Users\Medha\Desktop\c++> 

```

7. In the previous program make the data private, and functions public.

```

#include<string>
using namespace std;

class student {
private:
    string name;
    int roll;
    float marks;
public:

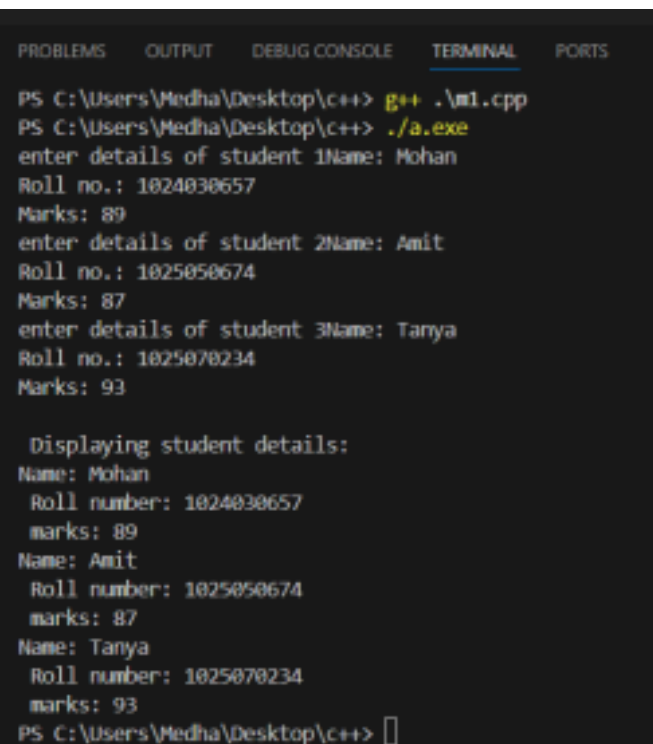
```

```

void setstudentData(string n, int r, float m)
{
    name = n;
    roll = r;
    marks = m; }
void getStudentData()
{
    cout << "Name: " << name << "\nRoll number: " << roll << "\nMarks: " << marks << endl;
};
int main()
{
    student s3[3];
    string n;
    int r;
    float m;
    for(int i = 0; i < 3; i++)
    {
        cout << "\nEnter details of student " << i + 1 << endl;
        cout << "Name: ";
        cin.ignore();          // clears buffer
        getline(cin, n);       // full name input
        cout << "Roll no.: ";
        cin >> r;
        cout << "Marks: ";
        cin >> m;
        s3[i].setstudentData(n, r, m);
    }
    cout << "\nDisplaying student details:\n";
    for(int i = 0; i < 3; i++)
    {
        s3[i].getStudentData();
        cout << endl;
    }
    return 0;
}

```

Output:



```

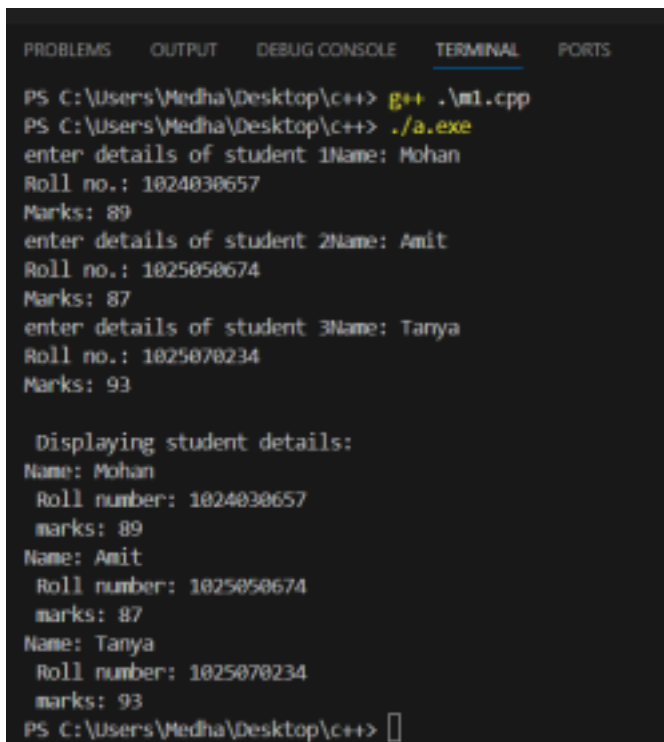
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Users\Medha\Desktop\c++> g++ .\m1.cpp
PS C:\Users\Medha\Desktop\c++> ./a.exe
enter details of student 1Name: Mohan
Roll no.: 1024030657
Marks: 89
enter details of student 2Name: Amit
Roll no.: 1025050674
Marks: 87
enter details of student 3Name: Tanya
Roll no.: 1025070234
Marks: 93

Displaying student details:
Name: Mohan
Roll number: 1024030657
marks: 89
Name: Amit
Roll number: 1025050674
marks: 87
Name: Tanya
Roll number: 1025070234
marks: 93
PS C:\Users\Medha\Desktop\c++> 

```

8. Convert program #4 from struct to class.

```
#include <iostream>
#include <string>
using namespace std;
class Student {
    string name;
    int roll;
    float marks;
public:
    void setStudentData(string n, int r, float m) {
        name = n;
        roll = r;
        marks = m; }
    void getStudentData() {
        cout << "Name: " << name << " Roll No: " << roll << " Marks: " << marks <<
endl; }
};
int main() {
    Student s3[3];
    string n;
    int r;
    float m;
    for (int i = 0; i < 3; i++) {
        cout << "Enter details of student " << i + 1 << endl;
        cout << "Name: ";
        cin >> n;
        cout << "Roll No: ";
        cin >> r;
        cout << "Marks: ";
        cin >> m;
        s3[i].setStudentData(n, r, m);
    }
    cout << "\nDisplaying student details:\n";
    for (int i = 0; i < 3; i++) {
        s3[i].getStudentData();
    } return 0;
}
```



The screenshot shows a terminal window with a dark background and light-colored text. At the top, there are tabs for 'PROBLEMS', 'OUTPUT', 'DEBUG CONSOLE', 'TERMINAL' (which is active), and 'PORTS'. The terminal shows the following sequence of commands and output:

```
PS C:\Users\Medha\Desktop\c++> g++ ./m1.cpp
PS C:\Users\Medha\Desktop\c++> ./a.exe
enter details of student 1Name: Mohan
Roll no.: 1024030657
Marks: 89
enter details of student 2Name: Amit
Roll no.: 1025050674
Marks: 87
enter details of student 3Name: Tanya
Roll no.: 1025070234
Marks: 93

Displaying student details:
Name: Mohan
Roll number: 1024030657
marks: 89
Name: Amit
Roll number: 1025050674
marks: 87
Name: Tanya
Roll number: 1025070234
marks: 93
PS C:\Users\Medha\Desktop\c++> 
```


9. Discuss the differences of structures in C and structures in C++ during the lab with other students or your instructor.

Feature	Structures in C	Structures in C++
Member Functions	Cannot have functions as members.	Can have functions (methods) as members.
Access Modifiers	Members are public by default; access modifiers (public, private, protected) are not supported.	Members are public by default, but access modifiers can be used.
Data Hiding	Not possible.	Possible using private or protected members.
Inheritance	Not supported.	Can inherit from other structures or classes.
struct Keyword	Required when declaring variables (e.g., struct Person p1;).	Optional when declaring variables after the type is defined (e.g., Person p1;).

10. WAP to read an integer value from the keyboard and display on screen “WELL DONE” that many times.

```
#include <iostream>
using namespace std;
int main() {
    int c;
    cout << "Enter an integer value: ";
    cin >> count;

    for (int i = 0; i < c; ++i) {
        std::cout << "WELL DONE" << endl;
    }

    return 0;
}
```

The screenshot shows a C++ IDE with a file named m2.cpp. The code in the editor is as follows:

```
1 #include <iostream>
2 using namespace std;
3 int main() {
4     int c;
5     cout << "Enter an integer value: ";
6     cin >> c;
7     for (int i = 0; i < c; ++i) {
8         cout << "WELL DONE" << endl;
9     }
10    return 0;
11 }
```

The terminal output shows the program running successfully. It prompts for an integer value, and the user enters 3. The program then outputs "WELL DONE" three times, once for each iteration of the loop.

```
collect2.exe: error: ld returned 1 exit status
PS C:\Users\Medha\Desktop\c++> g++ ./m2.cpp
PS C:\Users\Medha\Desktop\c++> .\a.exe
Enter an integer value: 3
WELL DONE
WELL DONE
WELL DONE
PS C:\Users\Medha\Desktop\c++>
```

LAB ASSIGNMENT 2

1. Create a structure in C++ containing the details of Students as details below and a main function to execute the structure.

Data Members(properties):

Name

Roll No

Degree

Hostel

Current CGPA

Member Function(behavior):

add Details();

updateDetails

updateCGPA();

update Hostel)

displaydetails();

CODE:

```
m3.cpp > student > updateHostel()
1  #include<iostream>
2  #include<string>
3  using namespace std;
4  struct student{
5      string name;
6      int roll;
7      string deg;
8      string hostel;
9      float cgpa;
10 void addDetails(string n, int r, string d,string h,float cg)
11 { name=n;
12   roll=r;
13   deg=d;
14   hostel=h;
15   cgpa=cg;
16 }
17 void updateDetails()
18 {
19     cout<<"\nenter new name:";
20     cin>>name;
21     cout<<"enter new roll no:";
22     cin>>roll;
23     cout<<"enter new degree:";
24     cin>>deg;
25 }
26 void updateCGPA()
27 {
28     cout << "Enter New CGPA: ";
29     cin >> cgpa;
30 }
31 void updateHostel()
32 {
33     cout << "Enter New Hostel: ";
34     cin >> hostel;
35 }
36 void displaydetails()
37 {cout<<"Name:"<<name<<"\n roll no.:"<< roll<<"\n Degree:"<<deg<<"\nhostel:"<<hostel<<"\n C
```

```

38     }
39 };
40 int main()
41 {
42     student s;
43     string n,h,d;
44     int r ;
45     float cg;
46     cout<< "enter details of student (name, roll no.,degree, hostel, cgpa)";
47     cin>>n>>r>>d>>h>>cg;
48     s.addDetails(n,r,d,h,cg);
49     s.displaydetails();
50     s.updateDetails();
51     s.updateCGPA();
52     s.displaydetails();
53     return 0;
54 }

```

Output

```

PS C:\Users\Medha\Desktop\c++> ./a.exe
enter details of student (name, roll no.,degree, hostel, cgpa)
Manya
102204096
BE
pg
8.2
Name:Manya
roll no.:102204096
Degree:BE
hostel:pg
Current CGPA:8.2
enter new name:Maanyaa
enter new roll no.:102303084
enter new degree:Btech
Enter New CGPA: 9.1
Name:Maanyaa
roll no.:102303084
Degree:Btech
hostel:pg
Current CGPA:9.1
PS C:\Users\Medha\Desktop\c++>

```

2.Differentiate between private and public access/scope. Perform the question no. 1 with class instead of structure with having the data members private and some member functions in private scope and some in public soope

Feature	Public Access	Private Access
Accessibility	Accessible from anywhere in the program.	Accessible only within the same class.
Visibility	Widely visible and part of the class's external interface.	Hidden from other classes and an internal implementation detail.
Encapsulation	Violates encapsulation if used for data members that need protection.	Essential for enforcing encapsulation and data hiding.

Code:

```
m3.cpp > ...
1  #include <iostream>
2  #include <string>
3  using namespace std;
4  class Student {
5  private:
6      string name;
7      int roll;
8      string degree;
9      string hostel;
10     float cgpa;
11     void updateDetails() {
12         cout << "\nEnter new name: ";
13         cin >> name;
14         cout << "Enter new roll no: ";
15         cin >> roll;
16         cout << "Enter new degree: ";
17         cin >> degree;
18     }
19 public:
20     void addDetails(string n, int r, string d, string h, float cg) {
21         name = n;
22         roll = r;
23         degree = d;
24         hostel = h;
25         cgpa = cg;
26     }
27     void updateCGPA() {
28         cout << "Enter new CGPA: ";
29         cin >> cgpa;
30     }
31     void updateHostel() {
32         cout << "Enter new hostel: ";
33         cin >> hostel;
34     }
35     // Public function calling private function
36     void modifyDetails() {
37         updateDetails();
38     }
39     void displayDetails() {
40         cout << "\nName: " << name
41             << "\nRoll No: " << roll
42             << "\nDegree: " << degree
43             << "\nHostel: " << hostel
44             << "\nCurrent CGPA: " << cgpa << endl;
45     };
46 int main() {
47     Student s;
48     string n, d, h;
49     int r;
50     float cg;
51     cout << "Enter details (name roll degree hostel cgpa): ";
52     cin >> n >> r >> d >> h >> cg;
53     s.addDetails(n, r, d, h, cg);
54     s.displayDetails();
55     s.modifyDetails(); // calls private updateDetails()
56     s.updateCGPA();
57     s.updateHostel();
58     s.displayDetails();
59     return 0;
60 }
```

Output:

```
Enter details (name roll degree hostel cgpa):
Ananya
1022150325
BE
Q
8.5

Name: Ananya
Roll No: 1022150325
Degree: BE
Hostel: Q
Current CGPA: 8.5

Enter new name: Aananyaa
Enter new roll no: 1023030366
Enter new degree: BTech
Enter new CGPA: 8.9
Enter new hostel: PG

Name: Aananyaa
Roll No: 1023030366
Degree: BTech
Hostel: PG
Current CGPA: 8.9
PS C:\Users\Medha\Desktop\c++> |
```

3.Create a code snippet that illustrates the following: Calling of private member functions inside public member function

```
opps > G L2_3.cpp > F
1  #include <iostream>
2  using namespace std;
3  class F {
4  private:
5      void privateFun() {
6          cout << "Private function called\n";
7      }
8  public:
9      void publicFun() {
10         privateFun();
11     };
12 int main() {
13     F d;
14     d.publicFun();
15     return 0;
16 }
17
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\Medha\Desktop\c++> g++ .\L2_3.cpp
PS C:\Users\Medha\Desktop\c++> ./a.exe
Private function called
PS C:\Users\Medha\Desktop\c++> |
```

4. Define a class Rectangle with variables width and height of integer type along with following:

a) void getdata() to initialize object values

b) void calculatearea() to calculate and display the area.

Code:

```
ops > L2_3.cpp > main
1  #include <iostream>
2  using namespace std;
3  class Rectangle {
4      int w, h;
5  public:
6      void getdata() {
7          cout << "Enter width and height: ";
8          cin >> w >> h;
9      }
10     void calculatearea() {
11         cout << "Area = " << w * h << endl;
12     }
13 };
14 int main() {
15     Rectangle r;
16     r.getdata();
17     r.calculatearea();
18     return 0;
19 }
20
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\Medha\Desktop\c++> g++ .\ops.\L2_3.cpp
PS C:\Users\Medha\Desktop\c++> ./a.exe
Enter width and height: 45 10
Area = 450
PS C:\Users\Medha\Desktop\c++> 
```

5. Define a class Complex with variables real and imaginary along with following

a) void setComplex (float, float) to initialize object values.

b) void displayComplex() to show the complex number

c) Pass and return objects to calculate sum of two complex numbers. Display the sum

Code:

```
m4.cpp > main()
1  #include<iostream>
2  using namespace std;
3  class complex {
4      float real;
5      float imag;
6      public:
7      void initialize(float r , float i)
8      {real =r;
9       imag =i;
10     }
11     void displaycomplex()
12     { cout<<real<<"+"<<imag<<"i"<<endl ;
13       }
14     void addcomplex( complex c1,complex c2 )
15     {complex temp;
16      temp.real= c1.real+c2.real;
17      temp.imag=c1.imag+c2.imag;
18      cout<<"Sum of 2 complex numners:";
19      cout<<temp.real<<"+"<<temp.imag<<"i";
20     }
21 };
22 int main()
23 {complex c1, c2, c3;
24  c1.initialize(3.5, 2.5);
25  c2.initialize(1.5, 4.5);
26
27  cout << "First Complex Number: ";
28  c1.displaycomplex();
29  cout << "Second Complex Number: ";
30  c2.displaycomplex();
31  c3.addcomplex(c1,c2);
32  return 0;
33 }
```

Output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\Medha\Desktop\c++> g++ .\m4.cpp
PS C:\Users\Medha\Desktop\c++> ./a.exe
First Complex Number: 3.5+2.5i
Second Complex Number: 1.5+4.5i
Sum of 2 complex numners:5+7i
PS C:\Users\Medha\Desktop\c++> 
```

6. Implement scope resolution operator: for the following uses:

(a) Class functions defined outside the class

```
1  #include<iostream>
2  using namespace std;
3  class Test {
4  public:
5      void show();
6  };
7  void Test::show() {
8      cout << "Function outside class\n";
9  }
10 int main(){
11     Test t;
12     t.show();
13     return 0;
14 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Function outside class

PS C:\Users\Medha\Desktop\c++> █

(b) Access a global variable with same name as a local variable

```
oops > L2_6.cpp > main()
1  #include<iostream>
2  using namespace std;
3  int x = 10;
4  int main() {
5      int x = 20;
6      cout << "Local x="<<x<<endl;
7      cout << "Global x="<<::x; // global x
8  }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\Medha\Desktop\c++> g++ .\oops.\L2_6.cpp

PS C:\Users\Medha\Desktop\c++> .\a.exe

Local x=20

Global x=10

PS C:\Users\Medha\Desktop\c++> █

(e) Access a static variables

```
oops > L2_6.cpp > a
1  #include<iostream>
2  using namespace std;
3  class C {
4  public:
5      static int a;
6  };
7  int C::a = 2;
8  int main() {
9      cout<<C::a<<endl;
10 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\Medha\Desktop\c++> g++ .\oops.\L2_6.cpp
PS C:\Users\Medha\Desktop\c++> .\a.exe
2
PS C:\Users\Medha\Desktop\c++> 
```

(d) Use inbuilt libraries functions

```
oops > L2_6.cpp > main()
1  #include<iostream>
2  int main() {
3      std::cout << "Medhachhavi Dhawan \n 1024150216";
4  }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\Medha\Desktop\c++> .\a.exe
Medhachhavi Dhawan
1024150216
PS C:\Users\Medha\Desktop\c++> 
```

7. Create a code to implement the namespace and use similar variables and functions defined in different code sections

```
oops > L2_6.cpp > ...
1  #include <iostream>
2  using namespace std;
3  namespace First {
4      int x = 10;
5      void show() {
6          cout << "First namespace\n";
7      }
8  }
9  namespace Second {
10     int x = 20;
11     void show() {
12         cout << "Second namespace\n";
13     }
14 }
15 int main() {
16     First::show();
17     cout << First::x << endl;
18     Second::show();
19     cout << Second::x << endl;
20     return 0;
21 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\Medha\Desktop\c++> g++ .\oops.\L2_6.cpp
PS C:\Users\Medha\Desktop\c++> ./a.exe
First namespace
10
Second namespace
20
PS C:\Users\Medha\Desktop\c++> |
```

LAB ASSIGNMENT 3

1. Write a program to implement (a) pointer to an object (b) this pointer. Practice both '.' (dot operator) and '->' (arrow operator).

Code:

```
oops > G+ L3.cpp > ...
1  #include <iostream>
2  using namespace std;
3  class Sample {
4      int x;
5  public:
6      void set(int x) {
7          this->x = x;    // this pointer
8      }
9      void show() {
10         cout << "Value of x = " << x << endl;
11     }
12 };
13
14 int main() {
15     Sample obj;
16     Sample *ptr;
17     ptr = &obj;
18     obj.set(10);        // using dot operator
19     obj.show();
20     ptr->set(20);        // using arrow operator
21     ptr->show();
22     return 0;
23 }
```

Output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Users\Medha\Desktop\c++> g++ .\oops.\L3.cpp
PS C:\Users\Medha\Desktop\c++> ./a.exe
Value of x = 10
Value of x = 20
PS C:\Users\Medha\Desktop\c++> |
```

2. Write a program to swap private values of two classes using a friend function.

Code:

```
oops > G+ L3.cpp > main()
1  #include <iostream>
2  using namespace std;
3  class B;
4  class A {
5      int x;
6  public:
7      A(int a) {
8          x = a;    }
9      friend void swapValues(A &, B &);
10     void show() {
11         cout << "A = " << x << endl;    }
12 };
13 class B {
14     int y;
15 public:
16     B(int b) {
17         y = b;    }
18     friend void swapValues(A &, B &);
19     void show() {
20         cout << "B = " << y << endl;    }
21 void swapValues(A &a, B &b) {
22     int temp = a.x;
23     a.x = b.y;
24     b.y = temp;}
```

Output:

```
PS C:\Users\Medha\Desktop\c++> g++ .\oops.\L3.cpp
PS C:\Users\Medha\Desktop\c++> ./a.exe
Before swapping:
A = 10
B = 20
After swapping:
A = 20
B = 10
PS C:\Users\Medha\Desktop\c++> |
```

```

25  int main() {
26      A obj1(10);
27      B obj2(20);
28      cout << "Before swapping:\n";
29      obj1.show();
30      obj2.show();
31      swapValues(obj1, obj2);
32      cout << "After swapping:\n";
33      obj1.show();
34      obj2.show();
35      return 0;
36  }

```

3. Write a program to add data objects of two different classes using friend functions.

Code:

```

oops > G+ L3.cpp > B
1  #include <iostream>
2  using namespace std;
3  class B;
4  class A {
5      int x;
6  public:
7      A(int a) { x = a; }
8      friend int add(A, B);
9  };
10 class B {
11     int y;
12 public:
13     B(int b) { y = b; }
14     friend int add(A, B);
15 };
16 int add(A a, B b) {
17     return (a.x + b.y);
18 }
19 int main() {
20     A a(5);
21     B b(15);
22     cout << "Sum = " << add(a, b) << endl;
23     return 0;
24 }

```

Output:

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL
PS C:\Users\Medha\Desktop\c++> g++ .\oops.L
PS C:\Users\Medha\Desktop\c++> ./a.exe
Sum = 20
PS C:\Users\Medha\Desktop\c++>

```

4. Write a program to demonstrate the working of friend class.

Code:

```
oops > G+ L3.cpp > A > A()
1  #include <iostream>
2  using namespace std;
3  class A {
4      int x;
5  public:
6      A() { x = 50; }
7      friend class B;
8  };
9  class B {
10 public:
11     void show(A obj) {
12         cout << "Value of x: " << obj.x << endl;
13     }
14 };
15 int main() {
16     A a;
17     B b;
18     b.show(a);
19     return 0;
20 }
```

Output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Users\Medha\Desktop\c++> g++ .\oops.\L3.c
PS C:\Users\Medha\Desktop\c++> ./a.exe
Value of x: 50
PS C:\Users\Medha\Desktop\c++> |
```

5. Write a program using Array of Objects to display area of multiple rectangles.

Code:

```
oops > G+ L3.cpp > main()
1  #include <iostream>
2  using namespace std;
3  class Rectangle {
4      float length, breadth;
5  public:
6      void setData(float l, float b) {
7          length = l;
8          breadth = b;
9      }
10     float area() {
11         return length * breadth;
12     }
13 };
14 int main() {
15     Rectangle r[3];
16     for(int i = 0; i < 3; i++) {
17         r[i].setData(i + 2, i + 3);
18         cout << "Area of rectangle " << i+1 << " = " << r[i].area() << endl;
19     }
20     return 0;
21 }
```

Output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Users\Medha\Desktop\c++> g++ .\oops.\L3.cpp
PS C:\Users\Medha\Desktop\c++> ./a.exe
Area of rectangle 1 = 6
Area of rectangle 1 = 6
Area of rectangle 1 = 6
Area of rectangle 2 = 12
Area of rectangle 3 = 20
PS C:\Users\Medha\Desktop\c++> |
```

6. Write a program to define function cube() as inline for calculating cube of a number.

Code:

```
oops > L3.cpp > ...
1  #include <iostream>
2  using namespace std;
3  inline int cube(int x) {
4      return x * x * x;
5  }
6  int main() {
7      int n;
8      cout << "Enter number: ";
9      cin >> n;
10     cout << "Cube = " << cube(n) << endl;
11     return 0;
12 }
```

Output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\Medha\Desktop\c++> g++ .\oops.\L3.cpp
PS C:\Users\Medha\Desktop\c++> ./a.exe
Enter number: 2
Cube = 8
PS C:\Users\Medha\Desktop\c++>
```

7. Write a program to pass an object as an argument and return the object from a function.

a. Use pass-by-value

Code:

```
oops > L3.cpp > ...
1  #include <iostream>
2  using namespace std;
3  class Number {
4  public:
5      int x;
6  };
7  Number increment(Number n) {
8      n.x = n.x + 1;
9      return n;
10 }
11 int main() {
12     Number n1, n2;
13     n1.x = 10;
14     n2 = increment(n1);
15     cout << "Original: " << n1.x << endl;
16     cout << "Returned: " << n2.x << endl;
17     return 0;
18 }
```

Output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\Medha\Desktop\c++> g++ .\oops.\L3.cpp
PS C:\Users\Medha\Desktop\c++> ./a.exe
Original: 10
Returned: 11
PS C:\Users\Medha\Desktop\c++> 
```

b. Use pass-by-address

Code:

```
oops > L3.cpp > main()
1  #include <iostream>
2  using namespace std;
3  class Number {
4  public:
5      int x;
6  };
7  Number increment(Number *n) {
8      n->x = n->x + 1;
9      return *n;
10 }
11 int main() {
12     Number n1, n2;
13     n1.x = 10;
14     n2 = increment(&n1);
15     cout << "Updated: " << n1.x << endl;
16     cout << "Returned: " << n2.x << endl;
17     return 0;
18 }
```

Output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL
PS C:\Users\Medha\Desktop\c++> g++ .\oops.L3.cpp
PS C:\Users\Medha\Desktop\c++> ./a.exe
Updated: 11
Returned: 11
PS C:\Users\Medha\Desktop\c++> 
```

LAB ASSIGNMENT4

1.Create a class named 'Rectangle' with two data members- length and breadth and a function to calculate the area which is 'length breadth'. The class has three constructors which are:

(a) having no parameter values of both length and breadth are assigned zero.

```
oops > L3.cpp > main()
3  class Rectangle{
5      float width;
6      public:
7      Rectangle()
8      {
9          lenght=1;
10         width=1;
11         cout<<"Default constructor callrd\n"
12     }
13     float area()
14     {
15         return lenght*width;
16     }
17     ~Rectangle(){
18         cout<<"Destructor called";
19     }
20 };
21 int main(){
22     Rectangle r1;
23     cout << "Area of r1: " << r1.area() << endl;
24 }
```

OUTPUT

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL

PS C:\Users\Medha\Desktop\c++> ./a.exe
Default constructor callrd
Area of r1: 1
Destructor called
PS C:\Users\Medha\Desktop\c++> 
```

(b) having two numbers as parameters - the two numbers are assigned as length and breadth respectively.

```
oops > L3.cpp > Rectangle > area()
3  class Rectangle{
7      Rectangle(float l,float w)
8      {
9          lenght=l;
10         width=w;
11     }
12 }
13 float area()
14 {
15     return lenght*width;
16 }
17 ~Rectangle(){
18     cout<<"Destructor called";
19 }
20 };
21 int main(){
22     Rectangle r1(4,5);
23     cout << "Area of r1: " << r1.area() << endl;
24 }
```

OUTPUT

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\Medha\Desktop\c++> g++ .\oops.\L3.cpp
PS C:\Users\Medha\Desktop\c++> ./a.exe
Area of r1: 20
Destructor called
PS C:\Users\Medha\Desktop\c++> 
```


(c) having one number as parameter both length and breadth are assigned that number.

Now, create objects of the 'Rectangle' class having none, one and two parameters and print their areas.

```
oops > L3.cpp > Rectangle > area()
3  class Rectangle{
7      Rectangle(float x) {
11     }
12
13     float area()
14     {
15         return length*width;
16     }
17     ~Rectangle(){
18         cout<<"Destructor called";
19     }
20 };
21 int main(){
22     Rectangle r1(4);
23     cout << "Area of r1: " << r1.area() << endl;
24 }
```

OUTPUT

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Users\Medha\Desktop\c++> g++ .\oops.\L3.cpp
PS C:\Users\Medha\Desktop\c++> ./a.exe
One-parameter constructor called
Area of r1: 16
Destructor called
PS C:\Users\Medha\Desktop\c++> |
```

2. Redefine the above program by creating an array of objects of the class Rectangle and calculate area for each object calling different constructors. Also implement constructors with default arguments and destructor in this program.

```
oops > L3.cpp > Rectangle > Rectangle(int, int)
1  #include <iostream>
2  using namespace std;
3  class Rectangle {
4      int length;
5      int breadth;
6  public:
7      Rectangle(int l = 1, int b = 1) {
8          length = l;
9          breadth = b;
10     }
11     int area() {
12         return length * breadth;
13     }
14     ~Rectangle() {
15         cout << "Destructor called " << endl;
16     }
17 };
18 int main() {
19     Rectangle rects[3] = { Rectangle(), Rectangle(5), Rectangle(4, 6) };
20     for (int i = 0; i < 3; i++) {
21         cout << "Area of Rectangle " << i+1 << ": " << rects[i].area() << endl;
22     }
23     return 0;
24 }
```

OUTPUT

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\Medha\Desktop\c++> g++ .\oops.\L3.cpp
PS C:\Users\Medha\Desktop\c++> ./a.exe
Area of Rectangle 1: 1
Area of Rectangle 2: 5
Area of Rectangle 3: 24
Destructor called
Destructor called
Destructor called
PS C:\Users\Medha\Desktop\c++> 
```

3. Verify the following about destructor by writing the program:

- (i) Name should begin with tilde sign(-) and must match class name.
- (ii) There cannot be more than one destructor in a class.
- (iii) Destructors do not allow any parameter.
- (iv) They do not have any return type, just like constructors.

```
oops > L3.cpp > Rectangle > ~Rectangle()
1  #include <iostream>
2  using namespace std;
3  class Rectangle {
4      int length;
5      int breadth;
6  public:
7      Rectangle(int l = 1, int b = 1) {
8          length = l;
9          breadth = b;
10     }
11     int area() {
12         return length * breadth;
13     }
14     ~Rectangle() {
15         cout << "Destructor called " << endl;
16     }
17 };
18 int main() { Rectangle r1;
19     Rectangle r2(5);
20     Rectangle r3(4, 6);
21     cout << "Area of r1: " << r1.area() << endl;
22     cout << "Area of r2: " << r2.area() << endl;
23     cout << "Area of r3: " << r3.area() << endl;
24     return 0;}
```

OUTPUT

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TE

PS C:\Users\Medha\Desktop\c++> g++ .\
PS C:\Users\Medha\Desktop\c++> ./a.ex
Area of r1: 1
Area of r2: 5
Area of r3: 24
Destructor called
Destructor called
Destructor called
PS C:\Users\Medha\Desktop\c++> 
```

4.Implement dynamic memory allocation. Use new and delete keywords.

(For integer variable, float variable, integer array, float array, class objects, Array of Objects)

```
oops > G L3.cpp > ...
1  #include <iostream>
2  using namespace std;
3  class Rectangle {
4      int length;
5      int breadth;
6  public:
7      Rectangle(int l = 0, int b = 0) {
8          length = l;
9          breadth = b;    }
10     int area() {
11         return length * breadth;    }
12     ~Rectangle() {
13         cout << "Destructor called for Rectangle" << endl;    };
14 int main() {
15     int* p = new int(3);
16     cout << "Integer value: " << *p << endl;
17     delete p;
18     float* f = new float(3.14);
19     cout << "Float value: " << *f << endl;
20     delete f;
21     int* arr = new int[5];
22     for (int i = 0; i < 5; i++) arr[i] = i + 1;
23     cout << "Integer array: ";
24     for (int i = 0; i < 5; i++) cout << arr[i] << " ";
25     cout << endl;
26     delete[] arr;
27     float* farr = new float[3];
28     for (int i = 0; i < 3; i++) farr[i] = i * 1.1;
29     cout << "Float array: ";
30     for (int i = 0; i < 3; i++) cout << farr[i] << " ";
31     cout << endl;
32     delete[] farr;
33     Rectangle* r = new Rectangle(5, 6);
34     cout << "Area of Rectangle: " << r->area() << endl;
35     delete r;
36     Rectangle* robj = new Rectangle[2]{ Rectangle(2, 3), Rectangle(4, 5) };
37     for (int i = 0; i < 2; i++) {
38         cout << "Area of Rectangle " << i+1 << ": " << robj[i].area() << endl;
39     }
40     delete[] robj;
41     return 0;
42 }
```

OUTPUT

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  F
PS C:\Users\Medha\Desktop\c++> ./a.exe
Integer value: 3
Float value: 3.14
Integer array: 1 2 3 4 5
Float array: 0 1.1 2.2
Area of Rectangle: 30
Destructor called for Rectangle
Area of Rectangle 1: 6
Area of Rectangle 2: 20
Destructor called for Rectangle
Destructor called for Rectangle
PS C:\Users\Medha\Desktop\c++> 
```

ASSIGNMENT:5

1. Write a simple base class, then a derived class and use objects of both of them in the main function. It will be a simple illustration of inheritance.

```
1  #include <iostream>
2  using namespace std;
3  class Base {
4  public:
5      void displayBase() {
6          cout << "This is Base class function." << endl;
7      }
8  };
9  class Derived : public Base {
10 public:
11     void displayDerived() {
12         cout << "This is Derived class function." << endl;
13     }
14 };
15 int main() {
16     Base obj1;
17     Derived obj2;
18     obj1.displayBase();
19     obj2.displayBase();
20     obj2.displayDerived();
21     return 0;
22 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\Medha\Desktop\c++> g++ .\oops.\Ao5.cpp
PS C:\Users\Medha\Desktop\c++> g++ .\oops.\Ao5.cpp
PS C:\Users\Medha\Desktop\c++> ./a.exe
This is Base class function.
This is Base class function.
This is Derived class function.
PS C:\Users\Medha\Desktop\c++> |
```

2. Practice protected access specifier in inheritance. In the base class declare a variable which is protected and access it in the derived class.

```
1  #include <iostream>
2  using namespace std;
3  class Base {
4  protected:
5      int num = 100;
6  };
7  class Derived : public Base {
8  public:
9      void show() {
10         cout << "Value of protected variable: " << num << endl;
11     }
12 };
13 int main() {
14     Derived obj;
15     obj.show();
16     return 0;
17 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\Medha\Desktop\c++> g++ .\oops.\Ao5.cpp
PS C:\Users\Medha\Desktop\c++> ./a.exe
Value of protected variable: 100
PS C:\Users\Medha\Desktop\c++> |
```

3. Most of the time we use public mode of inheritance, for example class Derived: public Base(); Try protected and private access modifiers to understand the difference of various modes of inheritance.



Public :

```
oops > Ao5.cpp > Derived
1  #include <iostream>
2  using namespace std;
3  class Base {
4  public:
5      int a = 10;
6  protected:
7      int b = 20;
8  };
9  class Derived : public Base {
10 public:
11     void show() {
12         cout << a << endl;
13         cout << b << endl;
14     }
15 };
16 int main() {
17     Derived obj;
18     cout << obj.a;
19 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\Medha\Desktop\c++> g++ .\oops.\Ao5.cpp
PS C:\Users\Medha\Desktop\c++> ./a.exe
10
PS C:\Users\Medha\Desktop\c++> █
```

Protected:

```
oops > Ao5.cpp > main()
1  #include <iostream>
2  using namespace std;
3  class Base {
4  public:
5      int a = 10;
6  };
7  class Derived : protected Base {
8  public:
9      void show() {
10         cout << a;
11     }
12 };
13 int main() {
14     Derived obj;
15     obj.show();
16     return 0;
17 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\Medha\Desktop\c++> g++ .\oops.\Ao5.cpp
PS C:\Users\Medha\Desktop\c++> ./a.exe
10
PS C:\Users\Medha\Desktop\c++> █
```

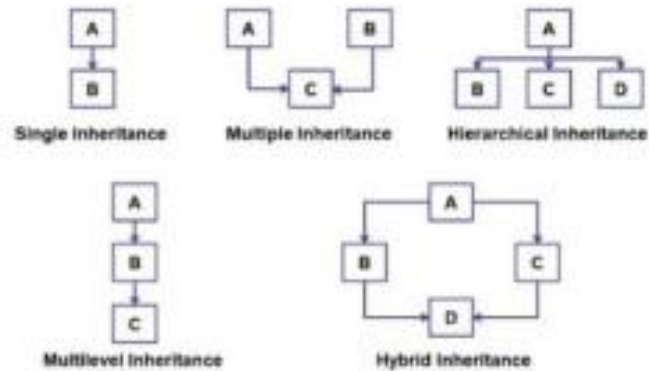
Private:

```
oops > Ao5.cpp > Base
1  #include <iostream>
2  using namespace std;
3  class Base {
4  public:
5      int a = 10;
6  };
7  class Derived : private Base {
8  public:
9      void show() {
10         cout << a;
11     }
12 };
13 int main() {
14     Derived obj;
15     obj.show();
16     return 0;
17 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\Medha\Desktop\c++> g++ .\oops.\Ao5.cpp
PS C:\Users\Medha\Desktop\c++> ./a.exe
10
PS C:\Users\Medha\Desktop\c++> █
```

4. Various types of inheritances are as shown below. Write small C++ codes for each inheritance type.



a) single inheritance

```
1  #include <iostream>
2  using namespace std;
3  class A {
4  public:
5      void showA() {
6          cout << "This is class A" << endl;
7      }
8  };
9  class B : public A {
10 public:
11     void showB() {
12         cout << "This is class B derived from A" << endl;
13     }
14 };
15 int main() {
16     B obj;
17     obj.showA();
18     obj.showB();
19     return 0;
20 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\Medha\Desktop\c++> g++ .\oops.\Ao5.cpp

PS C:\Users\Medha\Desktop\c++> ./a.exe

This is class A

This is class B derived from A

PS C:\Users\Medha\Desktop\c++> █

b) Multiple inheritance

```
oops > Ao5.cpp > main()
1  #include <iostream>
2  using namespace std;
3  class A {
4  public:
5      void showA() {
6          cout << "Class A function" << endl;
7      }
8  };
9  class B {
10 public:
11     void showB() {
12         cout << "Class B function" << endl;
13     }
14 };
15 class C : public A, public B {
16 public:
17     void showC() {
18         cout << "Class C derived from A and B" << endl;
19     }
20 };
21 int main() {
22     C obj;
23     obj.showA();
24     obj.showB();
25     obj.showC();
26 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\Medha\Desktop\c++> g++ .\oops.\Ao5.cpp
PS C:\Users\Medha\Desktop\c++> ./a.exe
This is class A
This is class B derived from A
PS C:\Users\Medha\Desktop\c++> |
```

c) Hierarchical inheritance

```
oops > Ao5.cpp > main()
1  #include <iostream>
2  using namespace std;
3  class A {
4  public:
5      void showA() {
6          cout << "This is base class A" << endl;
7      }
8  };
9  class B : public A {
```

```

9   class B : public A {
10  public:
11      void showB() {
12          cout << "Class B derived from A" << endl;
13      }
14  };
15  class C : public A {
16  public:
17      void showC() {
18          cout << "Class C derived from A" << endl;
19      }
20  };
21  int main() {
22      B obj1;
23      C obj2;
24      obj1.showA();
25      obj1.showB();
26      obj2.showA();
27      obj2.showC();
28  }

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\Medha\Desktop\c++> ./a.exe

This is class A

This is class B derived from A

PS C:\Users\Medha\Desktop\c++>

d)Multilevel inheritance

```

oops > G+ Ao5.cpp > B
1   #include <iostream>
2   using namespace std;
3   class A {
4   public:
5       void showA() {
6           cout << "Class A" << endl;
7       };
8   class B : public A {
9   public:
10      void showB() {
11          cout << "Class B derived from A" << endl;
12      };
13  class C : public B {
14  public:
15      void showC() {
16          cout << "Class C derived from B" << endl;
17      };
18  int main() {
19      C obj;
20      obj.showA();
21      obj.showB();
22      obj.showC();
23  }

```


PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\Medha\Desktop\c++> g++ .\oops.\Ao5.cpp
PS C:\Users\Medha\Desktop\c++> ./a.exe
This is class A
This is class B derived from A
PS C:\Users\Medha\Desktop\c++> |
```

e) Hybrid Inheritance

```
oops > Ao5.cpp > A
1  #include <iostream>
2  using namespace std;
3  class A {
4  public:
5      void showA() {
6          cout << "Class A" << endl;
7      };
8  class B : public A {
9  public:
10     void showB() {
11         cout << "Class B derived from A" << endl;
12     };
13     class C : public A {
14     public:
15         void showC() {
16             cout << "Class C derived from A" << endl;
17         };
18     class D : public B {
19     public:
20         void showD() {
21             cout << "Class D derived from B" << endl;
22         };
23     int main() {
24         D obj;
25         obj.showA();
26         obj.showB();
27         obj.showD();
28     }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\Medha\Desktop\c++> ./a.exe
Class A
Class B derived from A
Class D derived from B
PS C:\Users\Medha\Desktop\c++> |
```

5. How can you use constructors and destructors in C++ inheritance? Write a program to illustrate.

In C++ inheritance, constructors and destructors control how objects of base and derived classes are created and destroyed.

Order of Execution

- Constructor order
 1. Base class constructor executes first
 2. Derived class constructor executes second
- Destructor order
 1. Derived class destructor executes first
 2. Base class destructor executes second

This ensures the base part of an object is initialized before the derived part and cleaned up last.

```
oops > Ao5.cpp > main()
1  #include <iostream>
2  using namespace std;
3  class Base {
4  public:
5      Base() {
6          cout << "Base class constructor called" << endl;
7      }
8      ~Base() {
9          cout << "Base class destructor called" << endl;
10     }
11 };
12 class Derived : public Base {
13 public:
14     Derived() {
15         cout << "Derived class constructor called" << endl;
16     }
17     ~Derived() {
18         cout << "Derived class destructor called" << endl;
19     }
20 };
21 int main() {
22     Derived obj;
23     return 0;
24 }
```

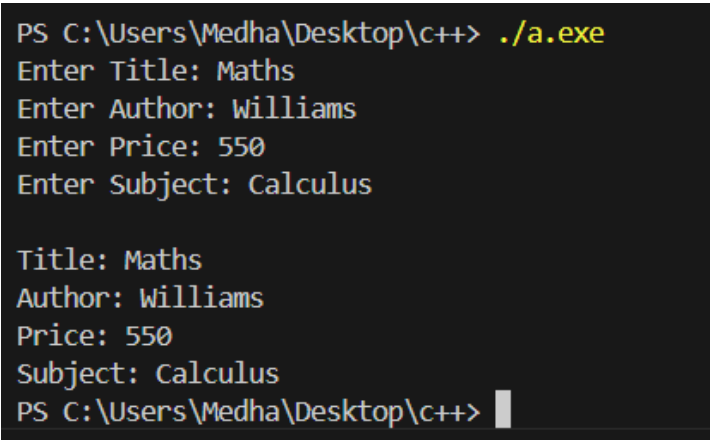
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\Medha\Desktop\c++> g++ .\Ao5.cpp
PS C:\Users\Medha\Desktop\c++> ./a.exe
Base class constructor called
Derived class constructor called
Derived class destructor called
Base class destructor called
PS C:\Users\Medha\Desktop\c++> 
```

6. Implement a base class Book with attributes title, author, and price. Then, create a derived class Textbook that inherits from Book and adds a new attribute subject.

Demonstrate how single inheritance is used to manage the data for general books and textbooks.

```
#include <iostream>
using namespace std;
class Book {
protected:
    string title;
    string author;
    float price;
public:
    void getBookData() {
        cout << "Enter Title: ";
        cin >> title;
        cout << "Enter Author: ";
        cin >> author;
        cout << "Enter Price: ";
        cin >> price;
    }
    void displayBookData() {
        cout << "\nTitle: " << title << endl;
        cout << "Author: " << author << endl;
        cout << "Price: " << price << endl;
    }
};
class Textbook : public Book {
private:
    string subject;
public:
    void getTextbookData() {
        getBookData();
        cout << "Enter Subject: ";
        cin >> subject;
    }
    void displayTextbookData() {
        displayBookData();
        cout << "Subject: " << subject <<
endl;
    };
};
int main() {
    Textbook t;
    t.getTextbookData();
    t.displayTextbookData();
    return 0;}
```



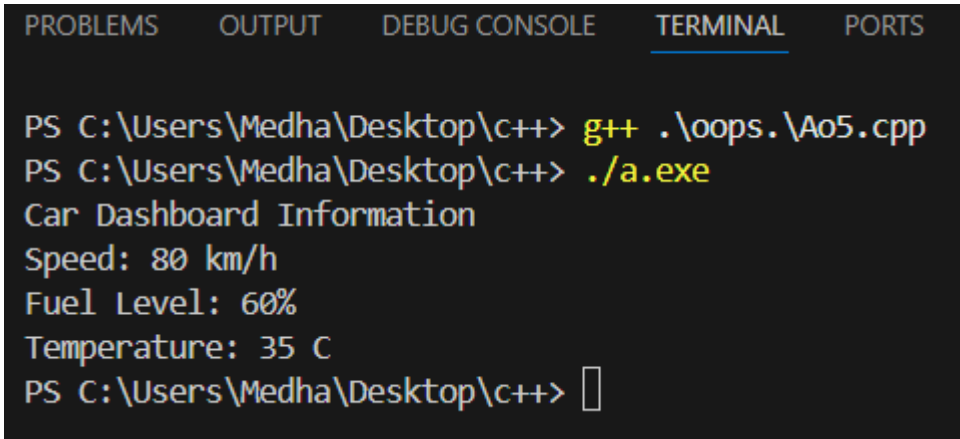
```
PS C:\Users\Medha\Desktop\c++> ./a.exe
Enter Title: Maths
Enter Author: Williams
Enter Price: 550
Enter Subject: Calculus

Title: Maths
Author: Williams
Price: 550
Subject: Calculus
PS C:\Users\Medha\Desktop\c++> |
```

7. A software company is creating a program to simulate a car's dashboard. The dashboard needs to display speed, fuel level, and temperature. The speed is controlled by a Speedometer class, the fuel level by a FuelGauge class, and the temperature by a Thermometer class. Implement the three classes: Speedometer, FuelGauge, and Thermometer, each with relevant attributes and methods. Then, create a CarDashboard class that inherits from all three classes to display the combined information on the dashboard. Demonstrate how multiple inheritance is used to build this class.

```
#include <iostream>
using namespace std;
class Speedometer {
protected:
    int speed;
public:
    void setSpeed(int s) {
        speed = s;
    }
};
class FuelGauge {
protected:
    int fuel;
public:
    void setFuel(int f) {
        fuel = f;
    }
};
class Thermometer {
protected:
    int temperature;
public:
    void setTemperature(int t) {
        temperature = t;
    }
};
class CarDashboard : public Speedometer, public FuelGauge, public Thermometer {
public:
    void displayDashboard() {
        cout << "Car Dashboard Information\n";
        cout << "Speed: " << speed << " km/h" << endl;
        cout << "Fuel Level: " << fuel << "%" << endl;
        cout << "Temperature: " << temperature << " C" << endl;
    }
};
```

```
int main() {
    CarDashboard car;
    car.setSpeed(80);
    car.setFuel(60);
    car.setTemperature(35);
    car.displayDashboard();
    return 0;
}
```



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\Medha\Desktop\c++> g++ .\oops.\Ao5.cpp
PS C:\Users\Medha\Desktop\c++> ./a.exe
Car Dashboard Information
Speed: 80 km/h
Fuel Level: 60%
Temperature: 35 C
PS C:\Users\Medha\Desktop\c++> █
```

8.You are tasked with creating a system for a library that tracks different types of users.The system needs to handle general user information such as name, ID, and contact details. There are two specific types of users: Student and Teacher. Each type of user has additional attributes, such as grade level for students and department for teachers. Implement a base class LibraryUser with general attributes. Then, create two derived classes Student and Teacher that inherit from LibraryUser and add their own specific attributes. Demonstrate how hierarchical inheritance is applied in this scenario.

```
#include <iostream>
using namespace std;
class LibraryUser {
protected:
    string name;
    int id;
    string contact;
public:
    void getUserData() {
        cout << "Enter Name: ";
        getline(cin, name);
        cout << "Enter ID: ";
        cin >> id;
        cout << "Enter Contact: ";
        cin >> contact;
    }
}
```

```

void displayUserData() {
    cout << "Name: " << name << endl;
    cout << "ID: " << id << endl;
    cout << "Contact: " << contact << endl;
};

class Student : public LibraryUser {
private:
    string grade;
public:
    void getStudentData() {
        getUserData();
        cout << "Enter Grade Level: ";
        getline(cin, grade); }
    void displayStudentData() {
        displayUserData();
        cout << "Grade Level: " << grade << endl;
    };
};

class Teacher : public LibraryUser {
private:
    string department;
public:
    void getTeacherData() {
        getUserData();
        cout << "Enter Department: ";
        getline(cin, department);
    }
    void displayTeacherData() {
        displayUserData();
        cout << "Department: " << department << endl;
    }
};

int main() {
    Student s;
    Teacher t;
    cout << "\nEnter Student Information\n";
    s.getStudentData();
    cout << "\nEnter Teacher Information\n";
    t.getTeacherData();
    cout << "\nStudent Details\n";
    s.displayStudentData();
    cout << "\nTeacher Details\n";
    t.displayTeacherData();
    return 0;
}

```

Output

```
PS C:\Users\Medha\Desktop\c++> ./a.exe
```

```
Enter Student Information
```

```
Enter Name: Medha
```

```
Enter ID: 1024150216
```

```
Enter Contact: 8273016159
```

```
Enter Grade Level: 2
```

```
Enter Teacher Information
```

```
Enter Name: Dr Anil
```

```
Enter ID: 41654
```

```
Enter Contact: 978546251
```

```
Enter Department: Electronics
```

```
Student Details
```

```
Name: Medha
```

```
ID: 1024150216
```

```
Contact: 8273016159
```

```
Grade Level: 2
```

```
Teacher Details
```

```
Name: Dr Anil
```

```
ID: 41654
```

```
Contact: 978546251
```

```
Department: Electronics
```

```
PS C:\Users\Medha\Desktop\c++> █
```

9. A logistics company needs a software system to manage its vehicle fleet. All vehicles share common attributes like make, model, and year. Trucks have additional attributes like load_capacity. Furthermore, refrigerated trucks have a special attribute called temperature_control. Implement a base class Vehicle with common attributes. Then, create a derived class Truck that adds the load_capacity attribute. Finally, create another derived class RefrigeratedTruck that inherits from Truck and adds the temperature_control attribute. Demonstrate how multilevel inheritance works in this case.

```
oops > Ao5.cpp > main()
1  #include <iostream>
2  using namespace std;
3  class Vehicle {
4  protected:
5      string make;
6      string model;
7      int year;
8  public:
9      Vehicle(string mk, string md, int yr) {
10         make = mk;
11         model = md;
12         year = yr;
13     }
14     void displayVehicle() {
15         cout << "Make: " << make << endl;
16         cout << "Model: " << model << endl;
17         cout << "Year: " << year << endl;
18     }
19 };
20 class Truck : public Vehicle {
21 protected:
22     int load_capacity;
23 public:
24     Truck(string mk, string md, int yr, int lc) : Vehicle(mk, md, yr) {
25         load_capacity = lc;
26     }
27     void displayTruck() {
28         displayVehicle();
29         cout << "Load Capacity: " << load_capacity << " tons" << endl;
30     }
31 };
32 class RefrigeratedTruck : public Truck {
33 private:
34     string temperature_control;
35 public:
36     RefrigeratedTruck(string mk, string md, int yr, int lc, string tc)
37         : Truck(mk, md, yr, lc) {
38         temperature_control = tc;
39     }
40     void displayRefrigeratedTruck() {
41         displayTruck();
42         cout << "Temperature Control: " << temperature_control << endl;
43     }
44 };
45 int main() {
46     RefrigeratedTruck rt("Volvo", "FH16", 2022, 20, "-5°C to 5°C");
47     cout << "Refrigerated Truck Details:" << endl;
48     rt.displayRefrigeratedTruck();
49     return 0;
50 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\Medha\Desktop\c++> ./a.exe
Refrigerated Truck Details:
Make: Volvo
Model: FH16
Year: 2022
Load Capacity: 20 tons
Temperature Control: -5°C to 5°C
PS C:\Users\Medha\Desktop\c++>
```

10. You are developing a software system for an academic institution. The institution has various roles like Person, Staff, and Student. A Person has general attributes like name and address. Staff members, who are a type of Person, have additional attributes like employee_id and department. Student, another type of Person, has attributes like student_id and grade. Some Staff members are also Students (e.g., teaching assistants) and need to inherit from both classes. Implement a base class Person with general attributes. Then, create derived classes Staff and Student that inherit from Person and add their specific attributes. Finally, create a TeachingAssistant class that inherits from both Staff and Student. Demonstrate how hybrid inheritance is applied and managed in this scenario.

```
oops > Ao5.cpp > Student > displayStudent()
1  #include <iostream>
2  using namespace std;
3  class Person {
4  protected:
5      string name;
6      string address;
7  public:
8      Person(string n, string a) {
9          name = n;
10         address = a;
11     }
12     void displayPerson() {
13         cout << "Name: " << name << endl;
14         cout << "Address: " << address << endl;
15     }
16 };
17 class Staff : public Person {
18 protected:
19     string employee_id;
20     string department;
21 public:
22     Staff(string n, string a, string eid, string dept)
23         : Person(n, a) {
24         employee_id = eid;
25         department = dept;
26     }
27     void displayStaff() {
28         displayPerson();
29         cout << "Employee ID: " << employee_id << endl;
30         cout << "Department: " << department << endl;
31     }
32 };
33 class Student : public Person {
34 protected:
35     string student_id;
36     char grade;
37 public:
```



```

37 public:
38     Student(string n, string a, string sid, char g)
39     : Person(n, a) {
40         student_id = sid;
41         grade = g;
42     }
43     void displayStudent() {
44         displayPerson();
45         cout << "Student ID: " << student_id << endl;
46         cout << "Grade: " << grade << endl;
47     }
48 class TeachingAssistant : public Staff, public Student {
49 public:
50     TeachingAssistant(string n, string a, string eid, string dept, string sid, char g)
51     : Staff(n, a, eid, dept), Student(n, a, sid, g) {}
52     void displayTA() {
53         cout << "Teaching Assistant Details:" << endl;
54         Staff::displayPerson(); // avoid ambiguity
55         cout << "Employee ID: " << employee_id << endl;
56         cout << "Department: " << department << endl;
57         cout << "Student ID: " << student_id << endl;
58         cout << "Grade: " << grade << endl;
59     }
60 int main() {
61     TeachingAssistant ta("Rahul Sharma", "Delhi", "EMP102", "Computer Science", "STU501", 'A');
62     ta.displayTA();
63     return 0;
64 }

```

PS C:\Users\Medha\Desktop\c++> g++ .\oops.\Ao5.cpp

PS C:\Users\Medha\Desktop\c++> ./a.exe

Teaching Assistant Details:

Name: Rahul Sharma

Address: Delhi

Employee ID: EMP102

Department: Computer Science

Student ID: STU501

Grade: A

PS C:\Users\Medha\Desktop\c++> █

ASSIGNMENT 6

1. Implement a class Library that contains an array of Book objects. The Book class should have attributes like title, author, and ISBN. The Library class should provide the following methods:

a. bool addNewBook(title, author, ISBN) to add the new book in the book list.

- The return type should be Boolean to check whether the book is added or not.
- This function should take the argument using pass-by-reference method.
- The attribute of the class (title, author, and ISBN) and the function arguments of function addNewBook should be same.
- Use the scope resolution operator to access the class data members.

b. bool removeBooks(ISBN) to remove the book using the input parameter ISBN.

- The return type should be Boolean to check whether the book is removed or not.
- This function also should take the argument using pass-by-reference method.
- Define function outside the class using scope resolution operator.

c. displayDetails() to list the books.

Write a main function to demonstrate adding, removing, and displaying books from the library. Add at least 5 books to demonstrate the adding book, 1 book to remove from the list.

```
#include <iostream>
using namespace std;
class Book {
public:
    string title;
    string author;
    string ISBN;
    Book(string t = "", string a = "", string i = "") {
        title = t;
        author = a;
        ISBN = i;
    }
};
class Library {
private:
    Book books[100];
    int count;
public:
    Library() {
        count = 0;
    }
    bool addNewBook(const Book &b);
    bool removeBooks(const string &isbn);
    void displayDetails();
};
```

```

bool Library::addNewBook(const Book &b) {
if (count >= 100) {
    return false;
}
books[count] = b;
count++;
return true;
}
bool Library::removeBooks(const string &isbn) {
    for (int i = 0; i < count; i++) {
        if (books[i].ISBN == isbn) {
            for (int j = i; j < count - 1; j++) {
                books[j] = books[j + 1];
            }
            count--;
            return true;
        }
    }
    return false;
}
void Library::displayDetails() {
    if (count == 0) {
        cout << "No books in library.\n";
        return;
    }
    cout << "\nLibrary Books:\n";
    for (int i = 0; i < count; i++) {
        cout << "Book " << i + 1 << ":\n";
        cout << "Title: " << books[i].title << endl;
        cout << "Author: " << books[i].author << endl;
        cout << "ISBN: " << books[i].ISBN << endl;
        cout << "-----\n";
    }
}
int main() {
    Library lib;
    lib.addNewBook(Book("C++ Basics", "Bjarne Stroustrup", "111"));
    lib.addNewBook(Book("Data Structures", "Mark Allen", "222"));
    lib.addNewBook(Book("Algorithms", "CLRS", "333"));
    lib.addNewBook(Book("Operating Systems", "Galvin", "444"));
    lib.addNewBook(Book("Computer Networks", "Tanenbaum", "555"));
}

```

```

cout << "After adding books:\n";
lib.displayDetails();
if (lib.removeBooks("333")) {
    cout << "\nBook with ISBN 333 removed successfully.\n";
} else {
    cout << "\nBook not found.\n";
}
cout << "\nAfter removing a book:\n";
lib.displayDetails();
return 0;
}

```

[Running] cd "c:\Users\Medha\Desktop\c++\oops\" && g++ A6.c
 After adding books:

```

Library Books:
Book 1:
Title: C++ Basics
Author: Bjarne Stroustrup
ISBN: 111
-----|

```

```

Book 2:
Title: Data Structures
Author: Mark Allen
ISBN: 222
-----

```

```

Book 3:
Title: Algorithms
Author: CLRS
ISBN: 333
-----

```

```

Book 4:
Title: Operating Systems
Author: Galvin
ISBN: 444
-----

```

```

Book 5:
Title: Computer Networks
Author: Tanenbaum
ISBN: 555
-----

```

Book with ISBN 333 removed successfully.

After removing a book:

Library Books:

Book 1:

Title: C++ Basics

Author: Bjarne Stroustrup

ISBN: 111

Book 2:

Title: Data Structures

Author: Mark Allen

ISBN: 222

Book 3:

Title: Operating Systems

Author: Galvin

ISBN: 444

Book 4:

Title: Computer Networks

Author: Tanenbaum

ISBN: 555

2. Implement the question 1 using constructor. Do the following modification:

- Add default constructor, parameterized constructor, and copy constructor to add the details of the book using the same parameters, i.e., title, author, and ISBN.
- Use the "this" pointer instead of scope resolution operator to access the data members of the class inside the constructor and function definition.
- Create function `bool removeBooks(ISBN)` same as the question 1.
- display `Details()` same as question 1.
- Create array of objects using 1. Initializer list, 2. Dynamic initialization

```
#include <iostream>
using namespace std;
class Book {
public:
    string title, author, ISBN;
    Book() {
        this->title = "Unknown";
        this->author = "Unknown";
        this->ISBN = "000";
    }
    Book(string title, string author, string ISBN) {
        this->title = title;
        this->author = author;
        this->ISBN = ISBN;
    }
    Book(const Book &b) {

        this->author = b.author;
```

```

        this->ISBN = b.ISBN;
    }
};

class Library {
private:
    Book books[100];
    int count;
public:
    Library() {
        this->count = 0;
    }
    bool addNewBook(const Book &b) {
        if (count >= 100) return false;
        books[count] = b;
        count++;
        return true;
    }
    bool removeBooks(const string &isbn) {
        for (int i = 0; i < count; i++) {
            if (books[i].ISBN == isbn) {
                for (int j = i; j < count - 1; j++) {
                    books[j] = books[j + 1];
                }
                count--;
                return true;
            }
        }
        return false;
    }
    void displayDetails() {
        cout << "\nLibrary Books:\n";
        for (int i = 0; i < count; i++) {
            cout << books[i].title << " | "
                 << books[i].author << " | "
                 << books[i].ISBN << endl;
        }
    }
};

int main() {
    Library lib;
    Book arr[3] = {
        Book("C++", "Bjarne", "101"),
        Book("DSA", "Mark", "102"),
        Book("OS", "Galvin", "103")
    };
};

```

```
for (int i = 0; i < 3; i++) {  
    lib.addNewBook(arr[i]);  
}  
Book *b1 = new Book("CN", "Tanenbaum", "104");  
Book *b2 = new Book("Algo", "CLRS", "105");  
lib.addNewBook(*b1);  
lib.addNewBook(*b2);  
cout << "Before Removal:\n";  
lib.displayDetails();  
lib.removeBooks("103");  
cout << "\nAfter Removal:\n";  
lib.displayDetails();  
delete b1;  
delete b2;  
return 0;  
}
```

```
[Running] cd "c:\Users\Medha\Desktop\c++\oops\" && g++ A6.cpp && ./A6.exe  
Before Removal:
```

Library Books:

```
C++ | Bjarne | 101  
DSA | Mark | 102  
OS | Galvin | 103  
CN | Tanenbaum | 104  
Algo | CLRS | 105
```

After Removal:

Library Books:

```
C++ | Bjarne | 101  
DSA | Mark | 102  
CN | Tanenbaum | 104  
Algo | CLRS | 105
```

3. Create a class Account with attributes account number (of const long type), transaction ID (of long type), transaction type (credit/debit) (of string type), and balance (of double type). The Account class should provide following methods:

- a. long depositAmount(to, from, amount).** The "to" and "from" are the account number of the sender and receiver. The function should return the transaction id.
- b. long creditAmount(to, from, amount)** similar to the depositAmount function. This function should also return the transaction id.
- c. void display Details()** should display all the details such as account number, remaining balance, transaction history (credited/debited). Make this function as the const function.
- d. In all the functions, make all the arguments as the const arguments.**

Write a main function to demonstrate all the function calls. Create at least 5 accounts objects (using array or without array) to demonstrate the operations of credit and debit.

Note: Use the scope resolution operator to include in-built libraries (cin, cout, endl, string etc.)

```
#include <iostream>
using namespace std;
class Account {
private:
    const long account_number;
    long transaction_id;
    string transaction_type;
    double balance;
public:
    Account(long acc_no, double bal)
        : account_number(acc_no), balance(bal) {
        transaction_id = 0;
        transaction_type = "None";
    }
    long depositAmount(const long &to, const long &from, const double &amount) {
        if (to == account_number) {
            balance += amount;
            transaction_id++;
            transaction_type = "Credited";
            return transaction_id;
        }
        return -1;
    }
    long creditAmount(const long &to, const long &from, const double &amount) {
```



```

        if (from == account_number && balance >= amount) {
            balance -= amount;
            transaction_id++;
            transaction_type = "Debited";
            return transaction_id;
        }
        return -1;
    }

    void displayDetails() const {
        cout << "Account No: " << account_number << endl;
        cout << "Balance: " << balance << endl;
        cout << "Last Transaction: " << transaction_type << endl;
        cout << "Transaction ID: " << transaction_id << endl;
        cout << "-----\n";
    }

    long getAccountNumber() const {
        return account_number;
    }
};

int main() {
    Account a1(1001, 5000);
    Account a2(1002, 4000);
    Account a3(1003, 3000);
    Account a4(1004, 2000);
    Account a5(1005, 1000);
    a1.creditAmount(a2.getAccountNumber(), a1.getAccountNumber(), 1000);
    a2.depositAmount(a2.getAccountNumber(), a1.getAccountNumber(), 1000);
    a3.creditAmount(a4.getAccountNumber(), a3.getAccountNumber(), 500);
    a4.depositAmount(a4.getAccountNumber(), a3.getAccountNumber(), 500);
    a2.creditAmount(a5.getAccountNumber(), a2.getAccountNumber(), 300);
    a5.depositAmount(a5.getAccountNumber(), a2.getAccountNumber(), 300);
    cout << "\nAccount Details:\n";
    a1.displayDetails();
    a2.displayDetails();
    a3.displayDetails();
    a4.displayDetails();
    a5.displayDetails();
    return 0;
}

```

```
[Running] cd "c:\Users\Medha\Desktop\c++\oops\" && g++  
"c:\Users\Medha\Desktop\c++\oops\tempCodeRunnerFile"
```

```
Account Details:  
Account No: 1001  
Balance: 4000  
Last Transaction: Debited  
Transaction ID: 1  
-----  
Account No: 1002  
Balance: 4700  
Last Transaction: Debited  
Transaction ID: 2  
-----  
Account No: 1003  
Balance: 2500  
Last Transaction: Debited  
Transaction ID: 1  
-----  
Account No: 1004  
Balance: 2500  
Last Transaction: Credited  
Transaction ID: 1  
-----  
Account No: 1005  
Balance: 1300  
Last Transaction: Credited  
Transaction ID: 1  
-----
```

4. Write a program to add data objects of two different classes using friend functions.

```
oops > A6.cpp > ClassB > ClassB(int)  
1  #include <iostream>  
2  using namespace std;  
3  class ClassB;  
4  class ClassA {  
5  private:  
6      int a;  
7  public:  
8      ClassA(int x) {  
9          a = x;  
10     }  
11     friend int add(ClassA, ClassB);  
12 };  
13 class ClassB {  
14 private:  
15     int b;  
16 public:  
17     ClassB(int y) {  
18         b = y;  
19     }  
20     friend int add(ClassA, ClassB);  
21 };
```

```

22     int add(ClassA objA, ClassB objB) {
23         return objA.a + objB.b;
24     }
25     int main() {
26         ClassA obj1(10);
27         ClassB obj2(20);
28         int result = add(obj1, obj2);
29         cout << "Sum = " << result << endl;
30         return 0;
31     }

```

OUTPUT

```

[Running] cd "c:\Users\Medha\Desktop\c++\oops\" && g++ tempCodeRu
"c:\Users\Medha\Desktop\c++\oops\"tempCodeRunnerFile
Sum = 30

[Done] exited with code=0 in 1.638 seconds

```

5. Define a class named Complex with properties (real and imaginary) and methods as per following details.

- Parameterized constructor and copy constructor to initialize object values.
- void display () to display complex number.
- void sum (Complex, Complex) to add two complex numbers (objects of Complex class). Implement this function as the friend function.

```

oops > A6.cpp > main()
1     #include <iostream>
2     using namespace std;
3     class Complex {
4     private:
5         int real, imag;
6     public:
7         Complex(int r, int i) {
8             real = r;
9             imag = i;
10        }
11        Complex(const Complex &c) {
12            real = c.real;
13            imag = c.imag;
14        }
15        void display() {
16            cout << real << " + " << imag << "i" << endl;
17        }
18        friend void sum(Complex, Complex);
19    };

```

```

20 void sum(Complex c1, Complex c2) {
21     int r = c1.real + c2.real;
22     int i = c1.imag + c2.imag;
23     cout << "Sum = " << r << " + " << i << "i" << endl;
24 }
25 int main() {
26     Complex c1(3, 4);
27     Complex c2(5, 6);
28     Complex c3 = c1;
29     cout << "Complex Number 1: ";
30     c1.display();
31     cout << "Complex Number 2: ";
32     c2.display();
33     cout << "Copied Complex Number: ";
34     c3.display();
35     sum(c1, c2);
36     return 0;
37 }

```

OUTPUT

```

[Running] cd "c:\Users\Medha\Desktop\c++\oops\" && g++ A6.cpp
Complex Number 1: 3 + 4i
Complex Number 2: 5 + 6i
Copied Complex Number: 3 + 4i
Sum = 8 + 10i

[Done] exited with code=0 in 1.456 seconds

```

ASSIGNMENT 7

1. Class polygon contains data member width and height and public method set_value() to assign values to width and height. Class Rectangle and Triangle are inherited from polygon class. Both the classes contain public method calculate_area() to calculate the area of Rectangle and Triangle. Use base class pointer to access the derived class object and show the area calculated.

```
1  #include <iostream>
2  using namespace std;
3  class Polygon {
4  protected:
5      float width, height;
6  public:
7      void set_value(float w, float h) {
8          width = w;
9          height = h;
10     }
11     virtual float calculate_area() = 0;
12 };
13 class Rectangle : public Polygon {
14 public:
15     float calculate_area() {
16         return width * height;
17     }
18 };
19 class Triangle : public Polygon {
20 public:
21     float calculate_area() {
22         return 0.5 * width * height;
23     }
24 };
25 int main() {
26     Polygon *p;
27     Rectangle r;
28     Triangle t;
29     r.set_value(10, 5);
30     t.set_value(10, 5);
31     p = &r;
32     cout << "Rectangle Area: " << p->calculate_area() << endl;
33     p = &t;
34     cout << "Triangle Area: " << p->calculate_area() << endl;
35     return 0;
36 }
```

OUTPUT

```
[Running] cd "c:\Users\Medha\Desktop\c++\oops\"
Rectangle Area: 50
Triangle Area: 25

[Done] exited with code=0 in 1.404 seconds
```

2. Write a program to create a class shape with functions area and display to find area and display the name of the shape and other essential component of the class. Create derived classes circle, rectangle and triangle each having overridden functions area and display. Write a program to find and display the area of circle, rectangle and triangle.

```
oops > G+ a7.cpp > main()
1  #include <iostream>
2  using namespace std;
3  class Shape {
4  public:
5      virtual void area() = 0;
6      virtual void display() = 0;
7  };
8  class Circle : public Shape {
9      float r;
10 public:
11     Circle(float r) { this->r = r; }
12     void area() {
13         cout << "Area of Circle: " << 3.14 * r * r << endl;
14     }
15     void display() {
16         cout << "Shape: Circle" << endl;
17     }
18 };
19 class Rectangle : public Shape {
20     float l, b;
21 public:
22     Rectangle(float l, float b) {
23         this->l = l;
24         this->b = b;
25     }
26     void area() {
27         cout << "Area of Rectangle: " << l * b << endl;
28     }
29     void display() {
30         cout << "Shape: Rectangle" << endl;
31     }
32 };
33 class Triangle : public Shape {
34     float b, h;
35 public:
36     Triangle(float b, float h) {
37         this->b = b;
38         this->h = h;
39     }
40     void area() {
41         cout << "Area of Triangle: " << 0.5 * b * h << endl;
42     }
43     void display() {
44         cout << "Shape: Triangle" << endl;
45     }
46 };
```

```

47 int main() {
48     Shape *s;
49     Circle c(5);
50     Rectangle r(4, 6);
51     Triangle t(3, 7);
52     s = &c; s->display(); s->area();
53     s = &r; s->display(); s->area();
54     s = &t; s->display(); s->area();
55     return 0;
56 }

```

OUTPUT

```

[Running] cd "c:\Users\Medha\Desktop\c++\oops\" && g++ a7.cpp
Shape: Circle
Area of Circle: 78.5
Shape: Rectangle
Area of Rectangle: 24
Shape: Triangle
Area of Triangle: 10.5

[Done] exited with code=0 in 1.402 seconds

```

3. Write a C++ program to compute area of right angle triangle, equilateral triangle, isosceles triangle using function overloading concept.

```

oops > g++ a7.cpp > main()
1  #include <iostream>
2  #include <cmath>
3  using namespace std;
4  double area(double base, double height) {
5      return 0.5 * base * height;
6  }
7  double area(double side) {
8      return (sqrt(3) / 4) * side * side;
9  }
10 double area(double base, double side1, bool isIsosceles) {
11     double height = sqrt(side1 * side1 - (base * base) / 4);
12     return 0.5 * base * height;
13 }
14 int main() {
15     double base, height, side;
16     cout << "Enter base and height of right triangle: ";
17     cin >> base >> height;
18     cout << "Area of right triangle = " << area(base, height) << endl;
19
20     cout << "\nEnter side of equilateral triangle: ";
21     cin >> side;
22     cout << "Area of equilateral triangle = " << area(side) << endl;
23     cout << "\nEnter base and equal side of isosceles triangle: ";
24     cin >> base >> side;
25     cout << "Area of isosceles triangle = " << area(base, side, true) << endl;
26     return 0;
27 }

```

OUTPUT

```
PS C:\Users\Medha\Desktop\c++> g++ .\oops.\a7.cpp
PS C:\Users\Medha\Desktop\c++> ./a.exe
Enter base and height of right triangle: 2 4
Area of right triangle = 4

Enter side of equilateral triangle: 4
Area of equilateral triangle = 6.9282

Enter base and equal side of isosceles triangle: 5
6
Area of isosceles triangle = 13.6359
PS C:\Users\Medha\Desktop\c++> █
```

4. Write a program with Student as abstract class and create derive classes Engineering, Medicine and Science from base class Student. Create the objects of the derived classes and process them and access them using array of pointer of type base class Student.

```
oops > a7.1.cpp > main()
1  #include <iostream>
2  using namespace std;
3  class Student {
4  public:
5      virtual void display() = 0;};
6  class Engineering : public Student {
7  public:
8      void display() {
9          cout << "Engineering Student" << endl;    };
10 class Medicine : public Student {
11 public:
12     void display() {
13         cout << "Medicine Student" << endl;    };
14 class Science : public Student {
15 public:
16     void display() {
17         cout << "Science Student" << endl;    };
18 int main() {
19     Student* s[3];
20     Engineering e;
21     Medicine m;
22     Science sc;
23     s[0] = &e;
24     s[1] = &m;
25     s[2] = &sc;
26     for(int i = 0; i < 3; i++) {
27         s[i]->display();
28     }
29     return 0;}
```

OUTPUT

```
[Running] cd "c:\Users\Medha\Desktop\c++\oops\" && g++ a7.1.c
Engineering Student
Medicine Student
Science Student

[Done] exited with code=0 in 1.552 seconds
```


5. Create a class Time with three private variables int h,m,s; Create a function to overload '+' operator to add two time variables.int main(){

```
Time t1(5,15,34),t2(9,53,58),t3;  
t3 = t1+t2;  
t3.show();  
}
```

```
oops > a7.1.cpp > Time  
1  #include <iostream>  
2  using namespace std;  
3  class Time {  
4  private:  
5      int h, m, s;  
6  public:  
7      Time(int h=0, int m=0, int s=0) {  
8          this->h = h;  
9          this->m = m;  
10         this->s = s;  
11     }  
12     Time operator + (Time t) {  
13         Time temp;  
14         temp.s = s + t.s;  
15         temp.m = m + t.m + temp.s / 60;  
16         temp.s %= 60;  
17         temp.h = h + t.h + temp.m / 60;  
18         temp.m %= 60;  
19         return temp;  
20     }  
21     void show() {  
22         cout << h << ":" << m << ":" << s << endl;  
23     }  
24 };  
25 int main() {  
26     Time t1(5,15,34), t2(9,53,58), t3;  
27     t3 = t1 + t2;  
28     t3.show();  
29     return 0;  
30 }
```

OUTPUT

```
[Running] cd "c:\Users\Medha\Desktop\c++\oops\" &  
15:9:32  
[Done] exited with code=0 in 1.402 seconds
```

6. Define a class STRING and overload == to compare two strings and + operator for concatenation of two strings.

```
oops > G a7.cpp > STRING > operator+(STRING)
1  #include <iostream>
2  #include <cstring>
3  using namespace std;
4  class STRING {
5      char str[100];
6  public:
7      STRING() {
8          str[0] = '\0';    }
9      void input() {
10         cin.getline(str, 100);    }
11     void display() {
12         cout << str;    }
13     bool operator==(STRING s) {
14         return strcmp(str, s.str) == 0;    }
15     STRING operator+(STRING s) {
16         STRING temp;
17         strcpy(temp.str, str);
18         strcat(temp.str, s.str);
19         return temp;    };
20 int main() {
21     STRING s1, s2, s3;
22     cout << "Enter first string: ";
23     s1.input();
24     cout << "Enter second string: ";
25     s2.input();
26     if (s1 == s2)
27         cout << "Strings are equal\n";
28     else
29         cout << "Strings are not equal\n";
30     s3 = s1 + s2;
31     cout << "Concatenated string: ";
32     s3.display();
33     return 0;
34 }
```

OYTPUT

```
PS C:\Users\Medha\Desktop\c++> g++ .\oops.\a7.cpp
PS C:\Users\Medha\Desktop\c++> ./a.exe
Enter first string: Medha
Enter second string: chhavi
Strings are not equal
Concatenated string: Medhachhavi
PS C:\Users\Medha\Desktop\c++> |
```

7. Write a program in C++ to create a class matrix and overload * operator using friend function to multiply two matrices.

```
oops > G+ a7.cpp > x operator*(Matrix, Matrix)
1  #include <iostream>
2  using namespace std;
3  class Matrix {
4      int a[2][2];
5  public:
6      void input() {
7          for(int i=0;i<2;i++)
8              for(int j=0;j<2;j++)
9                  cin >> a[i][j];      }
10     void display() {
11         for(int i=0;i<2;i++) {
12             for(int j=0;j<2;j++)
13                 cout << a[i][j] << " ";
14             cout << endl;          } }
15     friend Matrix operator*(Matrix m1, Matrix m2);};
16 Matrix operator*(Matrix m1, Matrix m2) {
17     Matrix result;
18     for(int i=0;i<2;i++) {
19         for(int j=0;j<2;j++) {
20             result.a[i][j] = 0;
21             for(int k=0;k<2;k++) {
22                 result.a[i][j] += m1.a[i][k] * m2.a[k][j]; } }
23     return result;
24 }
25 int main() {
26     Matrix m1, m2, m3;
27     cout << "Enter first matrix:\n";
28     m1.input();
29     cout << "Enter second matrix:\n";
30     m2.input();
31     m3 = m1 * m2;
32     cout << "Resultant matrix:\n";
33     m3.display();
34     return 0;
35 }
```

OUTPUT

```
PS C:\Users\Medha\Desktop\c++> g++ .\oops.\a7.cpp
PS C:\Users\Medha\Desktop\c++> ./a.exe
Enter first matrix:
1 2 3 4
Enter second matrix:
2 2 2 2
Resultant matrix:
6 6
14 14
PS C:\Users\Medha\Desktop\c++> 
```

8. Overload '[]' to check array index out of bounds problem at run time.

```
oops > a7.cpp > main()
1  #include <iostream>
2  using namespace std;
3  class Array {
4  |   int arr[5];
5  public:
6  |   void input() {
7  |       for(int i=0;i<5;i++)
8  |       |   cin >> arr[i];
9  |   }
10 |   int& operator[](int index) {
11 |       if(index < 0 || index >= 5) {
12 |           cout << "Error: Index out of bounds!\n";
13 |           exit(1);
14 |       }
15 |       return arr[index];
16 |   }
17 };
18 int main() {
19 |   Array a;
20 |   cout << "Enter 5 elements:\n";
21 |   a.input();
22 |   cout << "Access element at index 2: " << a[2] << endl;
23 |   cout << "Access element at invalid index 10:\n";
24 |   cout << a[10];
25 |   return 0;
26 }
```

Debug Console (Ctrl+Shift+Y)

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\Medha\Desktop\c++> g++ .\oops.\a7.cpp
PS C:\Users\Medha\Desktop\c++> ./a.exe
Enter 5 elements:
1 2 3 4 5
Access element at index 2: 3
Access element at invalid index 10:
Error: Index out of bounds!
PS C:\Users\Medha\Desktop\c++> |
```

9. Overload '()' to input arbitrary number of input arguments for an object.

```
oops > a7.cpp > main()
1  #include <iostream>
2  using namespace std;
3  class Input {
4  public:
5      void operator()(int a) {
6          std::ostream std::cout a << endl;
7      }
8      void operator()(int a, int b) {
9          cout << "Two values: " << a << " " << b << endl;
10     }
11     void operator()(int a, int b, int c) {
12         cout << "Three values: " << a << " " << b << " " << c << endl;
13     }
14 };
15 int main() {
16     Input obj;
17     obj(5);
18     obj(10, 20);
19     obj(1, 2, 3);
20     return 0;
21 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS 56

[Running] cd "c:\Users\Medha\Desktop\c++\oops\" && g++ a7.cpp -o a7 && "c:\Us
One value: 5
Two values: 10 20
Three values: 1 2 3

[Done] exited with code=0 in 1.63 seconds

10. Write a program in C++ to overload input operator (>>) and output operator (<<).

```
oops > a7.cpp > main()
1  #include <iostream>
2  using namespace std;
3  class Student {
4      int id;
5      float marks;
6  public:
7      friend istream& operator>>(istream &in, Student &s);
8      friend ostream& operator<<(ostream &out, Student &s);
9  };
10 istream& operator>>(istream &in, Student &s) {
11     cout << "Enter ID and Marks: ";
12     in >> s.id >> s.marks;
13     return in;
14 }
15 ostream& operator<<(ostream &out, Student &s) {
16     out << "ID: " << s.id << " Marks: " << s.marks << endl;
17     return out;
18 }
```

```

18 }
19 int main() {
20     Student s;
21     cin >> s;
22     cout << s;
23     return 0;
24 }

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

PS C:\Users\Medha\Desktop\c++> g++ .\oops.\a7.cpp
PS C:\Users\Medha\Desktop\c++> ./a.exe
Enter ID and Marks: 0216 95
ID: 216 Marks: 95
PS C:\Users\Medha\Desktop\c++> 

```

11. Write a program to convert basic data type (float) to user defined data type (object).

class Test {

private:

public:

Test(data_type) { // conversion code };

```

oops > a7.cpp > main()
1  #include <iostream>
2  using namespace std;
3  class Test {
4      float x;
5  public:
6      Test(float a) {
7          x = a;
8      }
9      void display() {
10         cout << "Value: " << x << endl;
11     };
12 int main() {
13     float num = 5.5;
14     Test t = num;
15     t.display();
16     return 0;
17 }

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

[Running] cd "c:\Users\Medha\Desktop\c++\oops\" &&
Value: 5.5

```

```

[Done] exited with code=0 in 1.425 seconds

```

12. Write a program to convert UDT to basic data type (float)

```
class Test {  
public:  
operator data type() { // conversion code }  
}
```

```
oops > G+ a7.cpp > Cartesian > show()  
1  #include <iostream>  
2  #include <cmath>  
3  using namespace std;  
4  class Cartesian;  
5  class Polar {  
6      float r, theta;  
7  public:  
8      Polar(float radius, float angle) {  
9          r = radius;  
10         theta = angle;    }  
11         operator Cartesian();};  
12  class Cartesian {  
13      float x, y;  
14  public:  
15      Cartesian() {}  
16      Cartesian(float a, float b) {  
17          x = a;  
18          y = b;    }  
19      void show() {  
20          cout << "x = " << x << " y = " << y << endl;    }  
21  Polar::operator Cartesian() {  
22      float x = r * cos(theta);  
23      float y = r * sin(theta);  
24      return Cartesian(x, y);}  
25  int main() {  
26      Polar p(10, 0.785);  
27      Cartesian c;  
28      c = p;  
29      c.show();  
30      return 0;}
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

56

[Running] cd "c:\Users\Medha\Desktop\c++\oops\" && g++ a7.cpp -o
x = 7.07388 y = 7.06825

[Done] exited with code=0 in 3.072 seconds

13. How will you convert one UDT to another UDT. For example conversion of polar to cartesian system.

Polar p(10.5);

Cartesian c = p;

c.show();

```
oops > G+ A8.cpp > main()
1  #include <iostream>
2  #include <cmath>
3  using namespace std;
4  class Cartesian;
5  class Polar {
6  |   float r, theta;
7  public:
8  |   Polar(float radius, float angle) {
9  |       r = radius;
10 |       theta = angle; }
11 |   operator Cartesian();};
12 class Cartesian {
13 |   float x, y;
14 public:
15 |   Cartesian() {}
16 |   Cartesian(float a, float b) {
17 |       x = a;
18 |       y = b; }
19 |   void show() {
20 |       cout << "x = " << x << " y = " << y << endl;
21 |   };
22 Polar::operator Cartesian() {
23 |   float x = r * cos(theta);
24 |   float y = r * sin(theta);
25 |   return Cartesian(x, y);}
26 int main() {
27 |   Polar p(10, 0.785);
28 |   Cartesian c;
29 |   c = p;
30 |   c.show();
31 |   return 0;}
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

[Running] cd "c:\Users\Medha\Desktop\c++\oops\" && g++ A8.cpp -o
x = 7.07388 y = 7.06825

[Done] exited with code=0 in 1.46 seconds

ASSIGNMENT-8: FUNCTION AND CLASS TEMPLATES

Function Template

1. Write a function template to swap two variables of any data type.

```
oops > A8.cpp > main()
1  #include <iostream>
2  using namespace std;
3  template <typename T>
4  void swapValues(T &a, T &b) {
5      T temp = a;
6      a = b;
7      b = temp;
8  }
9  int main() {
10     float a = 5, b = 10.5;
11     swapValues(a, b);
12     cout << "Swapped: \n" << "a=" << a << endl << "b=" << b << endl;
13     return 0;
14 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Filter (e.g. text)

[Running] cd "c:\Users\Medha\Desktop\c++\oops\" && g++ A8.cpp -o A8 &&
Swapped:
a=10.5
b=5

[Done] exited with code=0 in 1.458 seconds

2. Write a function template to find the minimum element in an array.

```
oops > A8.cpp > main()
1  #include <iostream>
2  using namespace std;
3  template <typename T>
4  T findmin(T a[],int s)
5  {
6      T min=a[0];
7      for(int i=0;i<s;i++)
8      {
9          if (a[i]<min)
10             min=a[i];
11      }
12      return min;
13  }
14 int main() {
15     int arr[] = {5, 2, 9, 1, 3};
16     int size = 5;
17     cout<<"minimum:"<<findmin(arr,size);
18     return 0;
19 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

[Running] cd "c:\Users\Medha\Desktop\c++\oops\" &&
minimum:1
[Done] exited with code=0 in 1.509 seconds

3. Write a function template to sort an array using bubble sort.

```
oops > G+ A8.cpp > bubblesort<T>(T [], int)
1  #include <iostream>
2  using namespace std;
3  template <typename T>
4  void bubblesort(T a[],int size){
5      for(int i=0;i<size-1;i++){
6          for (int j=0;j<size-i-1;j++){
7              if (a[j]>a[j+1]) {
8                  T temp=a[j];
9                  a[j]=a[j+1];
10                 a[j+1]=temp;
11             }
12         }
13     }
14     int main() {
15         float arr[]={1,5,23,78,12};
16         int size =5;
17         bubblesort(arr, size);
18         cout << "Sorted: ";
19         for (int i = 0; i < size; i++)
20             cout << arr[i] << " ";
21         cout << endl;
22     }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

[Running] cd "c:\Users\Medha\Desktop\c++\oops\
Sorted: 1 5 12 23 78

[Done] exited with code=0 in 1.543 seconds

4. Write a function template to perform linear search.

```
oops > G+ A8.cpp > main()
1  #include <iostream>
2  using namespace std;
3  template <typename T>
4  int linearSearch(T arr[], int size, T key) {
5      for (int i = 0; i < size; i++) {
6          if (arr[i] == key) {
7              return i;
8          }
9      }
10     return -1;
11 }
12 int main() {
13     int size;
14     cout << "Enter size of array: ";
15     cin >> size;
16     float arr[100];
17     cout << "Enter elements: ";
18     for (int i = 0; i < size; i++) {
19         cin >> arr[i];
20     }
21     float key;
22     cout << "Enter element to search: ";
23     cin >> key;
24     int index = linearSearch(arr, size, key);
25     if (index != -1)
26         cout << "Element found at index: " << index << endl;
27     else
28         cout << "Element not found" << endl;
29     return 0;
30 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\Medha\Desktop\c++> ./a.exe
Enter size of array: 5
Enter elements: 12 75 6 82 1
Enter element to search: 6
Element found at index: 2
PS C:\Users\Medha\Desktop\c++>

5. Design overloaded versions of a function template process() such that:

i) Accepts a single parameter

ii) Accepts two parameters of the same type

iii) Accepts two parameters of different types

```
oops > G+ A8.cpp > process<T, U>(T, U)
1  #include <iostream>
2  using namespace std;
3  template <typename T>
4  void process(T a) {
5      cout << "Single parameter: " << a << endl;
6  }
7  template <typename T>
8  void process(T a, T b) {
9      cout << "Two parameters (same type): " << a << " and " << b << endl;
10 }
11 template <typename T, typename U>
12 void process(T a, U b) {
13     cout << "Two parameters (different types): " << a << " and " << b << endl;
14 }
15 int main() {
16     int choice;
17     cout << "Choose option:\n";
18     cout << "1. Single parameter\n";
19     cout << "2. Two parameters (same type)\n";
20     cout << "3. Two parameters (different types)\n";
21     cout << "Enter choice: ";
22     cin >> choice;
23     if (choice == 1) {
24         int a;
25         cout << "Enter value: ";
26         cin >> a;
27         process(a);
28     }
29     else if (choice == 2) {
30         int a, b;
31         cout << "Enter two values (same type): ";
32         cin >> a >> b;
33         process(a, b);
34     }
35     else if (choice == 3) {
36         int a;
37         float b;
38         cout << "Enter int and float value: ";
39         cin >> a >> b;
40         process(a, b);
41     }
42     else {
43         cout << "Invalid choice";
44     }
45     return 0;
46 }
```

Class Template

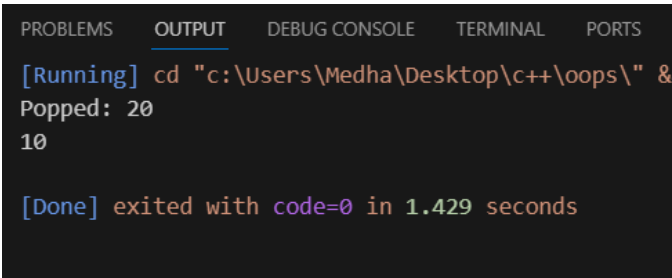
1. Develop a class template for stack with push and pop operations.

```
#include <iostream>
using namespace std;

template <class T>
class Stack {
    T arr[100];
    int top;
public:
    Stack() { top = -1; }
    void push(T x) {
        if (top == 99) {
            cout << "Stack Overflow\n";
            return;
        }
        arr[++top] = x;
    }
    void pop() {
        if (top == -1) {
            cout << "Stack Underflow\n";
            return;
        }
        cout << "Popped: " << arr[top--] << endl;
    }
    void display() {
        for (int i = top; i >= 0; i--)
            cout << arr[i] << " ";
        cout << endl;
    }
};

int main() {
    Stack<int> s;
    s.push(10);
    s.push(20);
    s.pop();
    s.display();
    return 0;
}
```

OUTOUT



The screenshot shows a terminal window with tabs for PROBLEMS, OUTPUT, DEBUG CONSOLE, TERMINAL, and PORTS. The OUTPUT tab is active, displaying the following text: [Running] cd "c:\Users\Medha\Desktop\c++\oops\" & Popped: 20 10 [Done] exited with code=0 in 1.429 seconds. The output matches the expected behavior of the program where the last pushed element (20) is popped and then the remaining element (10) is displayed.

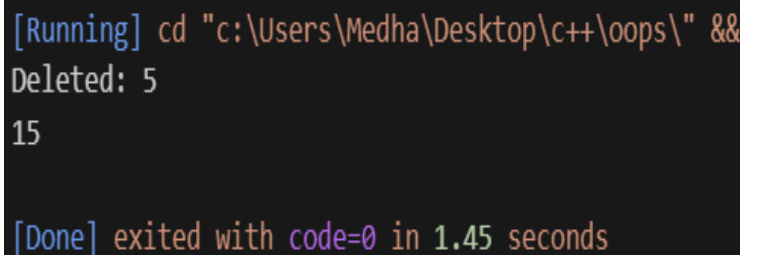
2. Develop a class template for queue with enqueue and dequeue operations.

```
#include <iostream>
using namespace std;
template <class T>
class Queue {
    T arr[100];
    int front, rear;
public:
    Queue() {
        front = rear = -1;
    }
    void enqueue(T x) {
        if (rear == 99) {
            cout << "Queue Overflow\n";
            return;
        }
        if (front == -1) front = 0;
        arr[++rear] = x;
    }
    void dequeue() {
        if (front == -1 || front > rear) {
            cout << "Queue Underflow\n";
            return;
        }
        cout << "Deleted: " << arr[front++] << endl;
    }

    void display() {
        for (int i = front; i <= rear; i++)
            cout << arr[i] << " ";
        cout << endl;
    }
};

int main() {
    Queue<int> q;
    q.enqueue(5);
    q.enqueue(15);
    q.dequeue();
    q.display();
    return 0;
}
```

OUTPUT



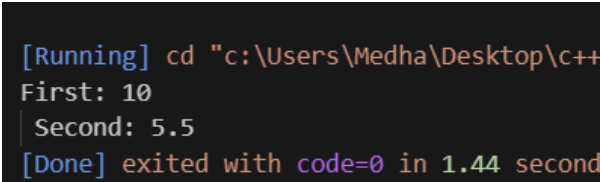
```
[Running] cd "c:\Users\Medha\Desktop\c++\oops\" &&
Deleted: 5
15
[Done] exited with code=0 in 1.45 seconds
```

3. Create a class template to store and display a pair of values.

```
#include <iostream>
using namespace std;
template <class T1, class T2>
class Pair {
    T1 a;
    T2 b;
public:
    Pair(T1 x, T2 y) {
        a = x;
        b = y;
    }
    void display() {
        cout << "First: " << a << endl << " Second: " << b ;
    }
};

int main() {
    Pair<int, float> p(10, 5.5);
    p.display();
    return 0;
}
```

OUTPUT



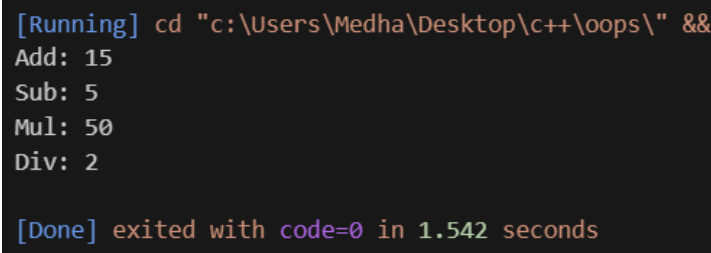
```
[Running] cd "c:\Users\Medha\Desktop\c++\" &&
First: 10
Second: 5.5
[Done] exited with code=0 in 1.44 seconds
```

4. Create a class template for basic arithmetic operations.

```
#include <iostream>
using namespace std;
template <class T>
class Arithmetic {
    T a, b;
public:
    Arithmetic(T x, T y) {
        a = x;
        b = y;
    }
    void add() { cout << "Add: " << a + b << endl; }
    void sub() { cout << "Sub: " << a - b << endl; }
    void mul() { cout << "Mul: " << a * b << endl; }
    void div() { cout << "Div: " << a / b << endl; };
};

int main() {
    Arithmetic<int> obj(10, 5);
    obj.add();
    obj.sub();
    obj.mul();
    obj.div();
    return 0;
}
```

OUTPUT



```
[Running] cd "c:\Users\Medha\Desktop\c++\oops\" &&
Add: 15
Sub: 5
Mul: 50
Div: 2
[Done] exited with code=0 in 1.542 seconds
```

5. Create a class template to input and display array elements.

```
#include <iostream>
using namespace std;
template <typename T>
class ArrayHandler {
    T arr[50];
    int size;
public:
    ArrayHandler(int n) : size(n) {}
    void input() {
        cout << "Enter " << size << " elements: ";
        for (int i = 0; i < size; i++) cin >> arr[i];
    }
    void display() const {
        cout << "Array: ";
        for (int i = 0; i < size; i++) cout << arr[i] << " ";
        cout << endl;
    }
};

int main() {
    ArrayHandler<int> ai(4);
    ArrayHandler<double> ad(3);
    ai.input(); ai.display();
    ad.input(); ad.display();
    return 0;
}
```

OUTPUT

```
PS C:\Users\Medha\Desktop\c++> g++ .\oops.\A8.cpp
PS C:\Users\Medha\Desktop\c++> ./a.exe
Enter 4 elements: 6 3 2 1
Array: 6 3 2 1
Enter 3 elements: 4.4 6.3 8.9
Array: 4.4 6.3 8.9
PS C:\Users\Medha\Desktop\c++> █
```

ASSIGNMENT 9

1. Write a C++ program to write number 1 to 200 in a data file NUM.TXT.

```
oops > a9.cpp > main()
1  #include<iostream>
2  #include<fstream>
3  using namespace std;
4  int main(){
5      ofstream fout("NUM.TXT");
6      if (!fout){
7          cout<<"Error opening file";
8          return 1;
9      }
10     for (int i=1;i<=200;i++)
11         fout<<i<<" ";
12     fout.close();
13     cout<<"numbers written successfully";
14     return 0;}

PROBLEMS OUTPUT ... Code
56

[Running] cd "c:\Users\Medha\Desktop\c++\oops\" && g++ a9.cpp
a9 && "c:\Users\Medha\Desktop\c++\oops\"a9
numbers written successfully
[Done] exited with code=0 in 1.534 seconds
```

2. Write a user-defined function in C++ to read the content from a text file NOTES.TXT, count and display the number of alphabets present in it.

```
oops > a9.cpp > countalpha()
1  #include<iostream>
2  #include<fstream>
3  using namespace std;
4  int countalpha(){
5      ifstream fin("C:\\Users\\Medha\\Desktop\\Notes.txt");
6      char ch;
7      int count=0;
8      if (!fin){
9          cout<<"error in opening file";
10         return 0; }
11     while (fin.get(ch)){
12         if (isalpha(ch))
13             count++;
14     }
15     fin.close();
16     return count;
17 }
18 int main(){
19     int result=countalpha();
20     cout<<"Number of alphabets:"<<result;
21     return 0;}

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
[Running] cd "c:\Users\Medha\Desktop\c++\oops\" && g++ a9.cpp -o
Number of alphabets:15
```

3. Write a program to copy the contents of one file to another.


```
oops > a9.cpp > main()
1  #include<iostream>
2  #include<fstream>
3  using namespace std;
4  int main(){
5      ifstream fin("C:\\Users\\Medha\\Desktop\\Notes.txt");
6      ofstream fout("destination_file.txt");
7      char ch;
8      if (!fin || !fout){
9          cout<<"Error in opening file";
10         return 1;
11     }
12     while(fin.get(ch)){
13         fout.put(ch);
14     }
15     fin.close();
16     fout.close();
17     cout<<"file copied successfully";
18     return 0;}
19

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  Filter (e.g. text)

[Running] cd "c:\\Users\\Medha\\Desktop\\c++\\oops\\" && g++ a9.cpp -o a
file copied successfully
[Done] exited with code=0 in 1.472 seconds
```

4. Write a program to implement I/O operations on characters. I/O operations include inputting a string, calculating the length of the string, storing the string in a file and fetching the stored characters from it.

```
oops > a9.cpp > main()
1  #include<iostream>
2  #include<fstream>
3  #include<cstring>
4  using namespace std;
5  int main(){
6      char str[100];
7      cout<<"enter a string";
8      cin.getline(str,100);
9      int length=strlen(str);
10     cout<<"length of string:"<<length<<endl;
11     ofstream fout("C:\\Users\\Medha\\Desktop\\Notes.txt");
12     fout<<str;
13     fout.close();
14     ifstream fin("C:\\Users\\Medha\\Desktop\\Notes.txt");
15     char ch;
16     cout<<"Stored charecter are:\n";
17     while(fin.get(ch))
18         cout<<ch;
19     fin.close();
20     return 0;
21 }

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\Medha\Desktop\c++> g++ .\oops.\a9.cpp
PS C:\Users\Medha\Desktop\c++> ./a.exe
enter a string Medhachhavi
length of string:13
Stored charecter are:
Medhachhavi
PS C:\Users\Medha\Desktop\c++> 
```

5. Write a program to do the followings:

(a) Create a file A-Z and read 10th character using seekg().

(b) Overwrite 5th number using seekp().

(c) Find file size using tellg().

(d) Read last character using seekg().

(e) Create a file data.txt with multiple lines. Read the file and:

i. Move the pointer to the 10th byte using seekg()

ii. Display the current position using tellg().

iii. Read and display the remaining content.

```
oops > g++ a9.cpp > main()
1  #include <iostream>
2  #include <fstream>
3  using namespace std;
4  int main() {
5      // (a) Create file A-Z
6      ofstream fout("alpha.txt");
7      for(char ch = 'A'; ch <= 'Z'; ch++) {
8          fout << ch;
9      }
10     fout.close();
11     ifstream fin1("alpha.txt");
12     fin1.seekg(9);
13     char ch;
14     fin1.get(ch);
15     cout << "10th character: " << ch << endl;
16     fin1.close();
17     // (b) Overwrite 5th character using seekp()
18     fstream file("alpha.txt", ios::in | ios::out);
19     file.seekp(4);
20     file.put('X');
21     file.close();
22     cout << "5th character overwritten with X" << endl;
23     ifstream fin2("alpha.txt");
24     fin2.seekg(0, ios::end);
25     int size = fin2.tellg();
26     cout << "File size: " << size << " bytes" << endl;
27     fin2.close();
28     // (d) Read last character using seekg()
29     ifstream fin3("alpha.txt");
30     fin3.seekg(-1, ios::end);
31     fin3.get(ch);
32     cout << "Last character: " << ch << endl;
33     fin3.close();
34     // (e) Create data.txt with multiple lines
35     ofstream fout2("data.txt");
36     fout2 << "This is line 1.\nThis is line 2.\nThis is line 3.";
37     fout2.close();
38     ifstream fin4("data.txt");
39     // (i) Move pointer to 10th byte
40     fin4.seekg(10);
41     // (ii) Display current position
42     cout << "Current position: " << fin4.tellg() << endl;
43     // (iii) Read remaining content
44     cout << "Remaining content:\n";
45     while(fin4.get(ch)) {
46         cout << ch;
47     }
48     fin4.close();
49     return 0;
}
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\Medha\Desktop\c++> g++ .\oops.a9.cpp
PS C:\Users\Medha\Desktop\c++> ./a.exe
enter a string Hello World!!
length of string:14
Stored charecter are:
Hello World!!
PS C:\Users\Medha\Desktop\c++> |
```

6. Write a program to do the followings:

i. Create a file with text "Hello World"

ii. After writing each character, display the current position of the put pointer using tellp().

iii. Replace "World" with "C++" using random access

```
oops > a9.cpp > main()
1  #include <iostream>
2  #include <fstream>
3  using namespace std;
4  int main() {
5      fstream file("hello.txt", ios::in | ios::out | ios::trunc);
6      file << "Hello World";
7      cout << "Position after writing: " << file.tellp() << endl;
8      file.seekp(6);
9      file << "C++ ";
10     file.seekg(0);
11     char ch;
12     cout << "Updated content:\n";
13     while (file.get(ch)) {
14         cout << ch;
15     }
16     file.close();
17     return 0;
18 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
[Running] cd "c:\Users\Medha\Desktop\c++\oops\" && g++ a9.cpp -o a9 && "c
Position after writing: 11
Updated content:
Hello C++
[Done] exited with code=0 in 1.298 seconds
```